Java 集合面试题

大家好，我是小林。

Java 集合是面试里几乎必考的一块，而且这块的题目有个特点：入门容易，但深挖起来没有底。很多人用 ArrayList 和 HashMap 用了好几年，但面试被问到"HashMap 为什么用红黑树而不是平衡二叉树""ConcurrentHashMap 在 JDK 1.7 和 1.8 的实现有什么区别"时，就开始说不清楚了。集合类看起来只是工具，但背后涉及的数据结构、哈希算法、并发机制，才是面试真正在考的东西。

这篇文章整理了 Java 集合框架面试中最常被问到的问题，涵盖 List、Set、Map 三大体系，重点包括 ArrayList 和 LinkedList 的差异、HashMap 的底层实现和扩容机制、ConcurrentHashMap 的线程安全方案，以及 HashSet、TreeMap、LinkedHashMap 这些常用实现类的原理和使用场景。

有几块内容在面试里被问得特别多，建议重点花时间：

- **HashMap**：put 过程、扩容机制、为什么初始容量是 2 的幂次方、链表转红黑树的条件，这些几乎每次都会问，而且很容易被追问到细节。
- **ConcurrentHashMap**：JDK 1.7 的分段锁和 JDK 1.8 的 CAS + synchronized 方案的区别，是并发面试里的高频考点。
- **ArrayList 的线程安全问题**：为什么不安全、具体会出现哪些问题、有哪些替代方案，这个看起来简单但答起来很容易流于表面。
- **equals 和 hashCode 的关系**：这两个方法为什么要配套重写，放到 HashMap 和 HashSet 里会有什么影响，基础但经常被忽略。

如果你是第一次系统准备这块，建议先搞清楚 ArrayList 和 HashMap 的底层原理，再去看线程安全相关的集合类，这样理解起来会顺很多。

概念

数组与集合区别，用过哪些?

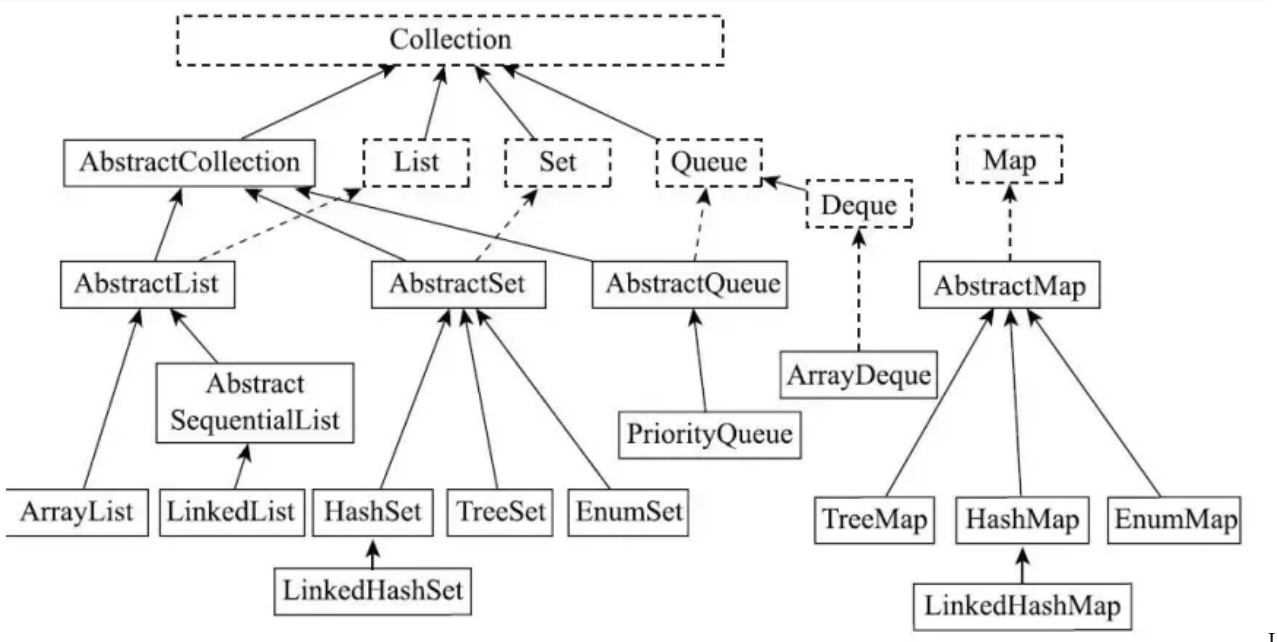
数组和集合的区别：

- 数组是固定长度的数据结构，一旦创建长度就无法改变，而集合是动态长度的数据结构，可以根据需要动态增加或减少元素。
- 数组可以包含基本数据类型和对象，而集合只能包含对象。
- 数组可以直接访问元素，而集合需要通过迭代器或其他方法访问元素。

我用过的一些 Java 集合类：

1. **ArrayList：** 动态数组，实现了 List 接口，支持动态增长。
2. **LinkedList：** 双向链表，也实现了 List 接口，支持快速的插入和删除操作。
3. **HashMap：** 基于哈希表的 Map 实现，存储键值对，通过键快速查找值。
4. **HashSet：** 基于 HashMap 实现的 Set 集合，用于存储唯一元素。
5. **TreeMap：** 基于红黑树实现的有序 Map 集合，可以按照键的顺序进行排序。
6. **LinkedHashMap：** 基于哈希表和双向链表实现的 Map 集合，保持插入顺序或访问顺序。
7. **PriorityQueue：** 优先队列，可以按照比较器或元素的自然顺序进行排序。

说说 Java 中的集合?



List 是有序的 Collection，使用此接口能够精确的控制每个元素的插入位置，用户能根据索引访问 List 中元素。常用的实现 List 的类有 LinkedList，ArrayList，Vector，Stack。

- ArrayList 是容量可变的非线程安全列表，其底层使用数组实现。当几何扩容时，会创建更大的数组，并把原数组复制到新数组。ArrayList 支持对元素的快速随机访问，但插入与删除速度很慢。
- LinkedList 本质是一个双向链表，与 ArrayList 相比，，其插入和删除速度更快，但随机访问速度更慢。

Set 不允许存在重复的元素，与 List 不同，set 中的元素是无序的。常用的实现有 HashSet，LinkedHashSet 和 TreeSet。

- HashSet 通过 HashMap 实现，HashMap 的 Key 即 HashSet 存储的元素，所有 Key 都是用相同的 Value，一个名为 PRESENT 的 Object 类型常量。使用 Key 保证元素唯一性，但不保证有序性。由于 HashSet 是 HashMap 实现的，因此线程不安全。
- LinkedHashSet 继承自 HashSet，通过 LinkedHashMap 实现，使用双向链表维护元素插入顺序。
- TreeSet 通过 TreeMap 实现的，添加元素到集合时按照比较规则将其插入合适的位置，保证插入后的集合仍然有序。

Map 是一个键值对集合，存储键、值和之间的映射。Key 无序，唯一；value 不要求有序，允许重复。Map 没有继承于 Collection 接口，从 Map 集合中检索元素时，只要给出键对象，就会返回对应的值对象。主要实现有 TreeMap、HashMap、HashTable、LinkedHashMap、ConcurrentHashMap

- HashMap：JDK1.8 之前 HashMap 由数组+链表组成的，数组是 HashMap 的主体，链表则是主要为了解决哈希冲突而存在的（“拉链法”解决冲突），JDK1.8 以后在解决哈希冲突时有了较大的变化，当链表长度大于阈值（默认为 8）时，将链表转化为红黑树，以减少搜索时间
- LinkedHashMap：LinkedHashMap 继承自 HashMap，所以它的底层仍然是基于拉链式散列结构即由数组和链表或红黑树组成。另外，LinkedHashMap 在上面结构的基础上，增加了一条双向链表，使得上面的结构可以保持键值对的插入顺序。同时通过对链表进行相应的操作，实现了访问顺序相关逻辑。
- HashTable：数组+链表组成的，数组是 HashTable 的主体，链表则是主要为了解决哈希冲突而存在的
- TreeMap：红黑树（自平衡的排序二叉树）
- ConcurrentHashMap：Node 数组+链表+红黑树实现，线程安全的（jdk1.8 以前 Segment 锁，1.8 以后 volatile + CAS 或者 synchronized）
-

Java 中的线程安全的集合是什么?

在 java.util 包中的线程安全的类主要 2 个，其他都是非线程安全的。

- **Vector**：线程安全的动态数组，其内部方法基本都经过 synchronized 修饰，如果不需要线程安全，并不建议选择，毕竟同步是有额外开销的。Vector 内部是使用对象数组来保存数据，可以根据需要自动的增加容量，当数组已满时，会创建新的数组，并拷贝原有数组数据。
- **Hashtable**：线程安全的哈希表，HashTable 的加锁方法是给每个方法加上 synchronized 关键字，这样锁住的是整个 Table 对象，不支持 null 键和值，由于同步导致的性能开销，所以已经很少被推荐使用，如果要保证线程安全的哈希表，可以用 ConcurrentHashMap。

java.util.concurrent 包提供的都是线程安全的集合：

并发 Map：

- **ConcurrentHashMap**：它与 HashTable 的主要区别是二者加锁粒度的不同，在 **JDK1.7**，ConcurrentHashMap 加的是分段锁，也就是 Segment 锁，每个 Segment 含有整个 table 的一部分，这样不同分段之间的并发操作就互不影响。在 **JDK 1.8** ，它取消了 Segment 字段，直接在 table 元素上加锁，实现对每一行进行加锁，进一步减小了并发冲突的概率。对于 put 操作，如果 Key 对应的数组元素为 null，则通过 CAS 操作（Compare and Swap）将其设置为当前值。如果 Key 对应的数组元素（也即链表表头或者树的根元素）不为 null，则对该元素使用 synchronized 关键字申请锁，然后进行操作。如果该 put 操作使得当前链表长度超过一定阈值，则将该链表转换为红黑树，从而提高寻址效率。
- **ConcurrentSkipListMap**：实现了一个基于 SkipList（跳表）算法的可排序的并发集合，SkipList 是一种可以在对数预期时间内完成搜索、插入、删除等操作的数据结构，通过维护多个指向其他元素的“跳跃”链接来实现高效查找。

并发 Set：

- **ConcurrentSkipListSet**：是线程安全的有序的集合。底层是使用 ConcurrentSkipListMap 实现。
- **CopyOnWriteArraySet**：是线程安全的 Set 实现，它是线程安全的无序的集合，可以将它理解成线程安全的 HashSet。有意思的是，CopyOnWriteArraySet 和 HashSet 虽然都继承于共同的父类 AbstractSet；但是，HashSet 是通过“散列表”实现的，而 CopyOnWriteArraySet 则是通过“动态数组(CopyOnWriteArrayList)”实现的，并不是散列表。

并发 List：

- **CopyOnWriteArrayList**：它是 ArrayList 的线程安全的变体，其中所有写操作（add，set 等）都通过对底层数组进行全新复制来实现，允许存储 null 元素。即当对象进行写操作时，使用了 Lock 锁做同步处理，内部拷贝了原数组，并在新数组上进行添加操作，最后将新数组替换掉旧数组；若进行的读操作，则直接返回结果，操作过程中不需要进行同步。

并发 Queue：

- **ConcurrentLinkedQueue**：是一个适用于高并发场景下的队列，它通过无锁的方式(CAS)，实现了高并发状态下的高性能。通常，ConcurrentLinkedQueue 的性能要好于 BlockingQueue 。
- **BlockingQueue**：与 ConcurrentLinkedQueue 的使用场景不同，BlockingQueue 的主要功能并不是在于提升高并发时的队列性能，而在于简化多线程间的数据共享。BlockingQueue 提供一种读写阻塞等待的机制，即如果消费者速度较快，则 BlockingQueue 则可能被清空，此时消费线程再试图从 BlockingQueue 读取数据时就会被阻塞。反之，如果生产线程较快，则 BlockingQueue 可能会被装满，此时，生产线程再试图向 BlockingQueue 队列装入数据时，便会被阻塞等待。

并发 Deque：

- **LinkedBlockingDeque**：是一个线程安全的双端队列实现。它的内部使用链表结构，每一个节点都维护了一个前驱节点和一个后驱节点。LinkedBlockingDeque 没有进行读写锁的分离，因此同一时间只能有一个线程对其进行操作
- **ConcurrentLinkedDeque**：ConcurrentLinkedDeque 是一种基于链接节点的无限并发链表。可以安全地并发执行插入、删除和访问操作。当许多线程同时访问一个公共集合时，ConcurrentLinkedDeque 是一个合适的选择。

Collections 和 Collection 的区别

- Collection 是 Java 集合框架中的一个接口，它是所有集合类的基础接口。它定义了一组通用的操作和方法，如添加、删除、遍历等，用于操作和管理一组对象。Collection 接口有许多实现类，如 List、Set 和 Queue 等。
- Collections（注意有一个 s）是 Java 提供的一个工具类，位于 java.util 包中。它提供了一系列静态方法，用于对集合进行操作和算法。Collections 类中的方法包括排序、查找、替换、反转、随机化等等。这些方法可以对实现了 Collection 接口的集合进行操作，如 List 和 Set。

集合遍历的方法有哪些?

在 Java 中，集合的遍历方法主要有以下几种：

- **普通 for 循环：** 可以使用带有索引的普通 for 循环来遍历 List。

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");

for (int i = 0; i < list.size(); i++) {
    String element = list.get(i);
    System.out.println(element);
}
```

- **增强 for 循环（for-each 循环）：** 用于循环访问数组或集合中的元素。

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");
for (String element : list) {
    System.out.println(element);
}
```

- **Iterator 迭代器：** 可以使用迭代器来遍历集合，特别适用于需要删除元素的情况。

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");
Iterator<String> iterator = list.iterator();
while(iterator.hasNext()) {
    String element = iterator.next();
    System.out.println(element);
}
```

- **ListIterator 列表迭代器：** ListIterator 是迭代器的子类，可以双向访问列表并在迭代过程中修改元素。

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");

ListIterator<String> listIterator= list.listIterator();
while(listIterator.hasNext()) {
    String element = listIterator.next();
    System.out.println(element);
}
```

- **使用 forEach 方法：** Java 8 引入了 forEach 方法，可以对集合进行快速遍历。

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");

list.forEach(element -> System.out.println(element));
```

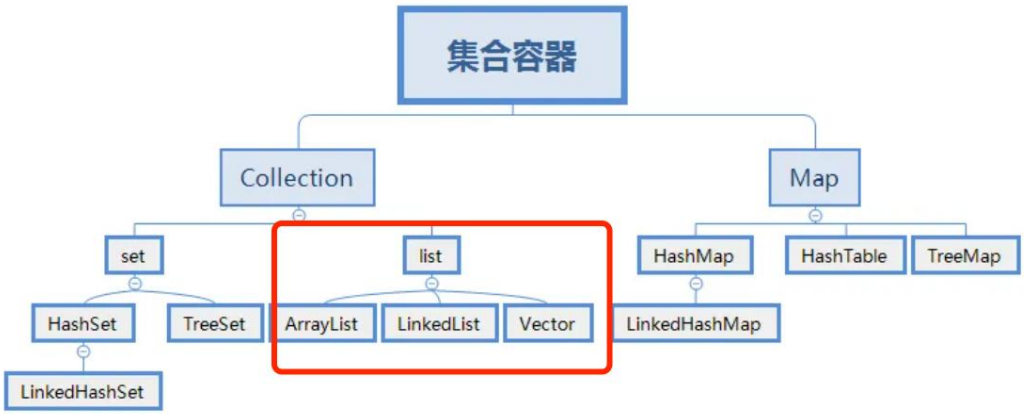
- **Stream API：** Java 8 的 Stream API 提供了丰富的功能，可以对集合进行函数式操作，如过滤、映射等。

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");

list.stream().forEach(element -> System.out.println(element));
```

这些是常用的集合遍历方法，根据情况选择合适的方法来遍历和操作集合。

List



常见的 List 集合（非线程安全）：

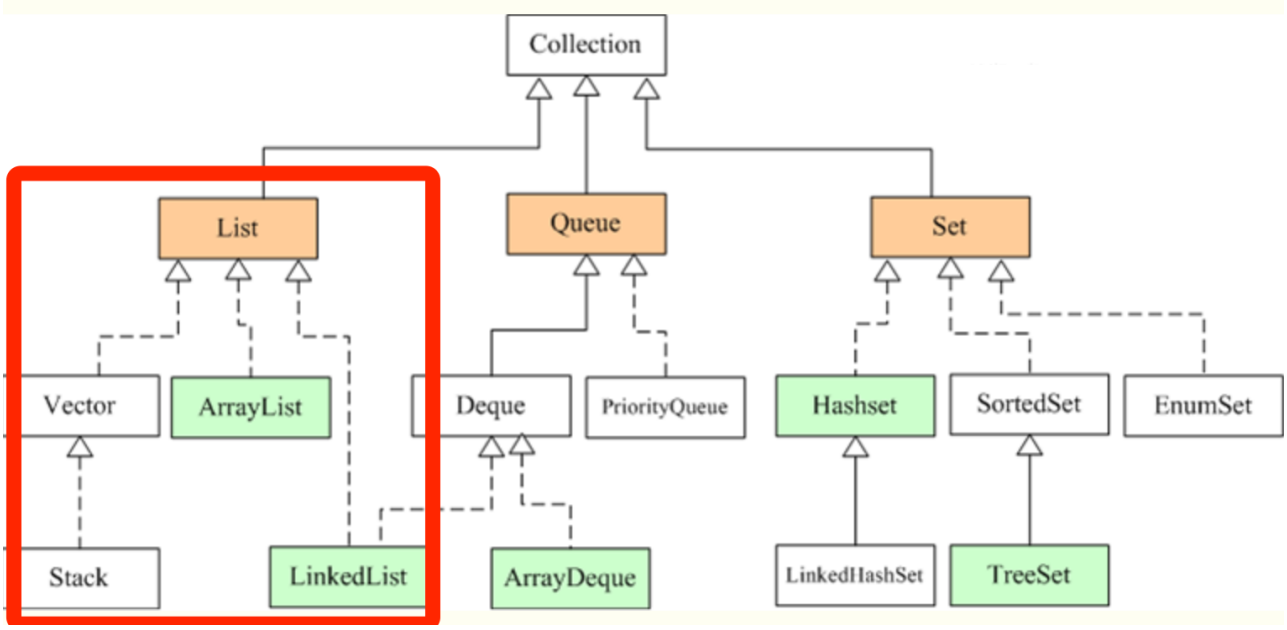
- `ArrayList` 基于动态数组实现，它允许快速的随机访问，即通过索引访问元素的时间复杂度为 O（1）。在添加和删除元素时，如果操作位置不是列表末尾，可能需要移动大量元素，性能相对较低。适用于需要频繁随机访问元素，而对插入和删除操作性能要求不高的场景，如数据的查询和展示等。
- `LinkedList` 基于双向链表实现，在插入和删除元素时，只需修改链表的指针，不需要移动大量元素，时间复杂度为 O（1）。但随机访问元素时，需要从链表头或链表尾开始遍历，时间复杂度为 O（n）。适用于需要频繁进行插入和删除操作的场景，如队列、栈等数据结构的实现，以及需要在列表中间频繁插入和删除元素的情况。

常见的 List 集合（线程安全）：

- `Vector` 和 `ArrayList` 类似，也是基于数组实现。`Vector` 中的方法大多是同步的，这使得它在多线程环境下可以保证数据的一致性，但在单线程环境下，由于同步带来的开销，性能会略低于 `ArrayList`。
- `CopyOnWriteArrayList` 在对列表进行修改（如添加、删除元素）时，会创建一个新的底层数组，将修改操作应用到新数组上，而读操作仍然在原数组上进行，这样可以保证读操作不会被写操作阻塞，实现了读写分离，提高了并发性能。适用于读操作远远多于写操作的并发场景，如事件监听列表等，在这种场景下可以避免大量的锁竞争，提高系统的性能和响应速度。

讲一下 java 里面 list 的几种实现，几种实现有什么不同?

在 Java 中，`List` 接口是最常用的集合类型之一，用于存储元素的有序集合。以下是 Java 中常见的 `List` 实现及其特点：



- Vector 是 Java 早期提供的线程安全的动态数组，如果不需要线程安全，并不建议选择，毕竟同步是有额外开销的。Vector 内部是使用对象数组来保存数据，可以根据需要自动的增加容量，当数组已满时，会创建新的数组，并拷贝原有数组数据。
- ArrayList 是应用更加广泛的动态数组实现，它本身不是线程安全的，所以性能要好很多。与 Vector 近似，ArrayList 也是可以根据需要调整容量，不过两者的调整逻辑有所区别，Vector 在扩容时会提高 1 倍，而 ArrayList 则是增加 50%。
- LinkedList 顾名思义是 Java 提供的双向链表，所以它不需要像上面两种那样调整容量，它也不是线程安全的。

> 这几种实现具体在什么场景下应该用哪种？

- Vector 和 ArrayList 作为动态数组，其内部元素以数组形式顺序存储的，所以非常适合随机访问的场合。除了尾部插入和删除元素，往往性能会相对较差，比如我们在中间位置插入一个元素，需要移动后续所有元素。
- 而 LinkedList 进行节点插入、删除却要高效得多，但是随机访问性能则要比动态数组慢。

list 可以一边遍历一边修改元素吗?

在 Java 中，`List` 在遍历过程中是否可以修改元素取决于遍历方式和具体的 `List` 实现类，以下是几种常见情况：

- 使用普通 for 循环遍历：可以在遍历过程中修改元素，只要修改的索引不超出 `List` 的范围即可。

```
import java.util.ArrayList;
import java.util.List;

public class ListTraversalAndModification {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);

        // 使用普通 for 循环遍历并修改元素
        for (int i = 0; i < list.size(); i++) {
            list.set(i, list.get(i) * 2);
        }

        System.out.println(list);
    }
}
```

- 使用 foreach 循环遍历：一般不建议在 `foreach` 循环中直接修改正在遍历的 `List` 元素，因为这可能会导致意外的结果或 `ConcurrentModificationException` 异常。在 `foreach` 循环中修改元素可能会破坏迭代器的内部状态，因为 `foreach` 循环底层是基于迭代器实现的，在遍历过程中修改集合结构，会导致迭代器的预期结构和实际结构不一致。

```
import java.util.ArrayList;
import java.util.List;

public class ListTraversalAndModification {
    public static void main(String[] args) {
```

```
        List<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);

        // 使用 foreach 循环遍历并尝试修改元素，会抛出 ConcurrentModificationException 异常
        for (Integer num : list) {
            list.set(list.indexOf(num), num * 2);
        }

        System.out.println(list);
    }
}
```

- 使用迭代器遍历时：可以使用迭代器的 $remove$ 方法来删除元素，但如果要**替换元素的值**，对于不可变对象（如 `Integer, String`），**必须**通过 `ListIterator` 的 `set` 方法来进行，而不是直接通过 `List` 的 `set` 方法，否则会抛出 `ConcurrentModificationException` 异常。

```
import java.util.ArrayList;
import java.util.ListIterator; // ⚠ 注意这里要用 ListIterator

public class ListTraversalAndModification {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);

        // // 使用 ListIterator 遍历并修改元素
        ListIterator<Integer> iterator = list.listIterator(); // ⚠ 使用 listIterator() 方法
        while (iterator.hasNext()) {
            Integer num = iterator.next();
            if (num.equals(2)) {
                // 使用 ListIterator 的 set 方法修改（替换）元素
                iterator.set(4);
```

```
            }
        }
        System.out.println(list); // 输出: [1, 4, 3]
    }
}
```

对于线程安全的 `List`，如 `CopyOnWriteArrayList`，由于其采用了写时复制的机制，在遍历的同时可以进行修改操作，不会抛出 `ConcurrentModificationException` 异常，但可能会读取到旧的数据，因为修改操作是在新的副本上进行的。

list 如何快速删除某个指定下标的元素?

`ArrayList` 提供了 `remove(int index)`方法来删除指定下标的元素，该方法在删除元素后，会将后续元素向前移动，以填补被删除元素的位置。如果删除的是列表末尾的元素，时间复杂度为 O（1）；如果删除的是列表中间的元素，时间复杂度为 O（n），n 为列表中元素的个数，因为需要移动后续的元素。示例代码如下：

```
import java.util.ArrayList;
import java.util.List;

public class ArrayListRemoveExample {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);

        // 删除下标为 1 的元素
        list.remove(1);

        System.out.println(list);
    }
}
```

`LinkedList` 的 `remove(int index)`方法也可以用来删除指定下标的元素。它需要先遍历到指定下标位置，然后修改链表的指针来删除元素。时间复杂度为 O（n），n 为要删除元素的下标。不过，如果

已知要删除的元素是链表的头节点或尾节点，可以直接通过修改头指针或尾指针来实现删除，时间复杂度为 O(1)。示例代码如下：

```
import java.util.LinkedList;
import java.util.List;

public class LinkedListRemoveExample {
    public static void main(String[] args) {
        List<Integer> list = new LinkedList<>();
        list.add(1);
        list.add(2);
        list.add(3);

        // 删除下标为 1 的元素
        list.remove(1);

        System.out.println(list);
    }
}
```

`CopyOnWriteArrayList` 的 `remove` 方法同样可以删除指定下标的元素。由于 `CopyOnWriteArrayList` 在写操作时会创建一个新的数组，所以删除操作的时间复杂度取决于数组的复制速度，通常为 O(n)，n 为数组的长度。但在并发环境下，它的删除操作不会影响读操作，具有较好的并发性能。示例代码如下：

```
import java.util.concurrent.CopyOnWriteArrayList;

public class CopyOnWriteArrayListRemoveExample {
    public static void main(String[] args) {
        CopyOnWriteArrayList<Integer> list = new CopyOnWriteArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);

        // 删除下标为 1 的元素
        list.remove(1);
```

```
        System.out.println(list);
    }
}
```

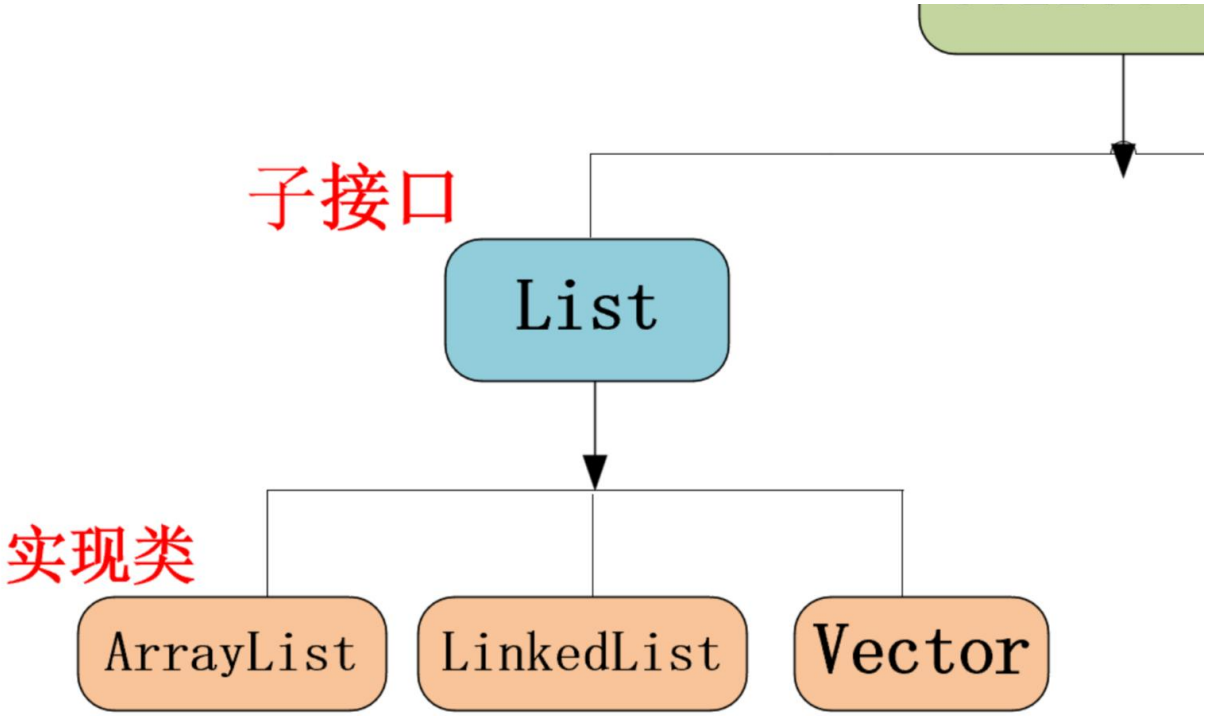
Arraylist 和 LinkedList 的区别，哪个集合是线程安全的?

ArrayList 和 LinkedList 都是 Java 中常见的集合类，它们都实现了 List 接口。

- **底层数据结构不同**：ArrayList 使用动态数组实现，通过索引可以快速定位到元素。LinkedList 使用双向链表实现，每个节点都存储了元素本身以及指向前一个和后一个节点的指针，通过节点之间的指针关联来访问和操作元素。
- **插入和删除操作的效率不同**：ArrayList 在尾部进行插入和删除操作时效率较高，因为不需要移动其他元素；但如果是在中间或开头插入、删除，就需要移动后面的所有元素，效率会比较低。LinkedList 在头部和尾部进行插入、删除操作时效率很高，只需要调整节点的指针即可；但如果是在中间位置操作，需要先从头或尾遍历链表找到目标位置，时间复杂度也是 O (n)，不过找到位置后只需要调整指针，不需要像 ArrayList 那样移动大量元素，所以在某些特定场景下还是有优势的，而且 LinkedList 实现了 Deque 接口，还可以当作双端队列、栈来使用。
- **随机访问的效率不同**：ArrayList 支持通过索引直接快速访问元素，时间复杂度为 O (1)。LinkedList 不支持随机访问，想要获取某个位置的元素，必须从头节点或尾节点开始逐个遍历，时间复杂度为 O (n)。
- **空间占用**：ArrayList 在创建时会分配一段连续的内存空间，虽然会有一定的容量浪费（比如实际元素没装满数组），但只需要存储元素本身。LinkedList 每个节点除了存储元素，还需要额外存储两个指针（指向前一个和后一个节点），所以在存储相同数量元素的情况下，LinkedList 的空间占用通常会比 ArrayList 更大一些。
- **使用场景**：ArrayList 更适合需要频繁随机访问元素，或者主要在尾部进行插入、删除操作的场景。LinkedList 更适合需要频繁在头部或尾部进行插入、删除操作，或者需要作为双端队列、栈使用的场景；如果是通过迭代器直接操作已知位置的节点，在中间插入、删除时也能发挥它调整指针快的优势。
- **线程安全**：这两个集合都不是线程安全的，如果在多线程环境下使用，需要自己加锁保证线程安全，或者使用线程安全的 List 集合，比如 Vector、Collections.synchronizedList () 包装的 List，或者 CopyOnWriteArrayList。

arraylist 和 vector 区别是什么?

ArrayList 和 Vector 都是 Java 中常用的动态数组实现，用于存储和操作对象集合，但它们在设计上有几个关键区别，主要体现在线程安全性、性能和功能细节上。



首先是线程安全性，这是最核心的区别。Vector 是线程安全的，它的大部分方法（比如 add、remove、get 等）都被 synchronized 修饰，这意味着多线程环境下操作 Vector 时，不需要额外处理同步问题。而 ArrayList 没有任何同步机制，是非线程安全的，在多线程并发修改时可能会出现数据不一致的问题，比如抛出 ConcurrentModificationException 异常。

正因为同步机制的存在，两者在性能上也有差异。由于 Vector 的方法需要加锁释放锁，在单线程环境下，它的操作效率通常比 ArrayList 低。所以如果是单线程场景，或者能自己保证线程安全的情况下，ArrayList 是更优的选择，性能更好。

另外，在扩容机制上，两者也有所不同。当集合元素数量超过初始容量时，都会自动扩容。Vector 默认的扩容策略是翻倍（如果没有指定增长因子的话），比如初始容量 10，满了之后会扩容到 20。而 ArrayList 在 JDK 1.8 及之后，默认是扩容为原来的 1.5 倍，相对来说扩容的幅度更小，能在一定程度上节省内存空间。当然，Vector 也可以通过构造方法指定增长因子，灵活控制扩容幅度，而 ArrayList 没有这个功能。

总的来说，选择两者时主要看是否需要线程安全：如果是多线程环境且需要内置同步支持，可能会用到 Vector；但现在更多时候会用 ArrayList，因为它性能更好，而且在需要线程安全时，可以通过 Collections.synchronizedList () 方法将 ArrayList 包装成线程安全的集合，灵活性更高。

ArrayList 线程安全吗？把 ArrayList 变成线程安全有哪些方法?

不是线程安全的，ArrayList 变成线程安全的方式有：

- 使用 Collections 类的 synchronizedList 方法将 ArrayList 包装成线程安全的 List：

```
List<String> synchronizedList = Collections.synchronizedList(arrayList);
```

- 使用 CopyOnWriteArrayList 类代替 ArrayList，它是一个线程安全的 List 实现：

```
CopyOnWriteArrayList<String> copyOnWriteArrayList = new CopyOnWriteArrayList<>(arrayList);
```

- 使用 Vector 类代替 ArrayList，Vector 是线程安全的 List 实现：

```
Vector<String> vector = new Vector<>(arrayList);
```

为什么 ArrayList 不是线程安全的，具体来说是哪里不安全?

在高并发添加数据下，ArrayList 会暴露三个问题;

- 部分值为 null（我们并没有 add null 进去）
- 索引越界异常
- size 与我们 add 的数量不符

为了知道这三种情况是怎么发生的，ArrayList，add 增加元素的代码如下：

```
public boolean add(E e) {
        ensureCapacityInternal(size + 1);  // Increments modCount!!
        elementData[size++] = e;
        return true;
    }
```

ensureCapacityInternal()这个方法的详细代码我们可以暂时不看，它的作用就是判断如果将当前的新元素加到列表后面，列表的 elementData 数组的大小是否满足，如果 size + 1 的这个需求长度大于了 elementData 这个数组的长度，那么就要对这个数组进行扩容。

大体可以分为三步：

- 判断数组需不需要扩容，如果需要的话，调用 grow 方法进行扩容；
- 将数组的 size 位置设置值（因为数组的下标是从 0 开始的）；
- 将当前集合的大小加 1

下面我们来分析三种情况都是如何产生的：

- 部分值为 null：当线程 1 走到了扩容那里发现当前 size 是 9，而数组容量是 10，所以不用扩容，这时候 cpu 让出执行权，线程 2 也进来了，发现 size 是 9，而数组容量是 10，所以不用扩容，这时候线程 1 继续执行，将数组下标索引为 9 的位置 set 值了，还没有来得及执行 size++，这时候线程 2 也来执行了，又把数组下标索引为 9 的位置 set 了一遍，这时候两个先后进行 size++，导致下标索引 10 的地方就为 null 了。
- 索引越界异常：线程 1 走到扩容那里发现当前 size 是 9，数组容量是 10 不用扩容，cpu 让出执行权，线程 2 也发现不用扩容，这时候数组的容量就是 10，而线程 1 set 完之后 size++，这时候线程 2 再进来 size 就是 10，数组的大小只有 10，而你要设置下标索引为 10 的就会越界（数组的下标索引从 0 开始）；
- size 与我们 add 的数量不符：这个基本上每次都会发生，这个理解起来也很简单，因为 size++本身就不是原子操作，可以分为三步：获取 size 的值，将 size 的值加 1，将新的 size 值覆盖掉原来的，线程 1 和线程 2 拿到一样的 size 值加完了同时覆盖，就会导致一次没有加上，所以肯定不会与我们 add 的数量保持一致的；

ArrayList 和 LinkedList 的应用场景?

- ArrayList 适用于需要频繁访问集合元素的场景。它基于数组实现，可以通过索引快速访问元素，因此在按索引查找、遍历和随机访问元素的操作上具有较高的性能。当需要频繁访问和遍历集合元素，并且集合大小不经常改变时，推荐使用 ArrayList
- LinkedList 适用于频繁进行插入和删除操作的场景。它基于链表实现，插入和删除元素的操作只需要调整节点的指针，因此在插入和删除操作上具有较高的性能。当需要频繁进行插入和删除操作，或者集合大小经常改变时，可以考虑使用 LinkedList。

ArrayList 的扩容机制说一下

ArrayList 在添加元素时，如果当前元素个数已经达到了内部数组的容量上限，就会触发扩容操作。ArrayList 的扩容操作主要包括以下几个步骤：

- 计算新的容量：一般情况下，新的容量会扩大为原容量的 1.5 倍（在 JDK 10 之后，扩容策略做了调整），然后检查是否超过了最大容量限制。

- 创建新的数组：根据计算得到的新容量，创建一个新的更大的数组。
- 将元素复制：将原来数组中的元素逐个复制到新数组中。
- 更新引用：将 ArrayList 内部指向原数组的引用指向新数组。
- 完成扩容：扩容完成后，可以继续添加新元素。

ArrayList 的扩容操作涉及到数组的复制和内存的重新分配，所以在频繁添加大量元素时，扩容操作可能会影响性能。为了减少扩容带来的性能损耗，可以在初始化 ArrayList 时预分配足够大的容量，避免频繁触发扩容操作。

之所以扩容是 1.5 倍，是因为 1.5 可以充分利用移位操作，减少浮点数或者运算时间和运算次数。

```
// 新容量计算
int newCapacity = oldCapacity + (oldCapacity >> 1);
```

线程安全的 List， CopyonWriteArraylist 是如何实现线程安全的

CopyOnWriteArrayList 底层也是通过一个数组保存数据，使用 volatile 关键字修饰数组，保证当前线程对数组对象重新赋值后，其他线程可以及时感知到。

```
private transient volatile Object[] array;
```

在写入操作时，加了一把互斥锁 ReentrantLock 以保证线程安全。

```
public boolean add(E e) {
    //获取锁
    final ReentrantLock lock = this.lock;
    //加锁
    lock.lock();
    try {
        //获取到当前 List 集合保存数据的数组
        Object[] elements = getArray();
        //获取该数组的长度（这是一个伏笔，同时 len 也是新数组的最后一个元素的索引值）
        int len = elements.length;
        //将当前数组拷贝一份的同时，让其长度加 1
        Object[] newElements = Arrays.copyOf(elements, len + 1);
```

```
            //将加入的元素放在新数组最后一位，len 不是旧数组长度吗，为什么现在用它当成新数组的最后一个元素的下标？建议自行画图推演，就很容易理解。
            newElements[len] = e;
            //替换引用，将数组的引用指向给新数组的地址
            setArray(newElements);
            return true;
        } finally {
            //释放锁
            lock.unlock();
        }
    }
```

看到源码可以知道写入新元素时，首先会先将原来的数组拷贝一份并且让原来数组的长度+1 后就得到了一个新数组，新数组里的元素和旧数组的元素一样并且长度比旧数组多一个长度，然后将新加入的元素放置都在新数组最后一个位置后，用新数组的地址替换掉老数组的地址就能得到最新的数据了。

在我们执行替换地址操作之前，读取的是老数组的数据，数据是有效数据；执行替换地址操作之后，读取的是新数组的数据，同样也是有效数据，而且使用该方式能比读写都加锁要更加的效率。

现在我们来看读操作，读是没有加锁的，所以读是一直都能读

```
public E get(int index) {
    return get(getArray(), index);
}
```

List<>里面填基本数据类型为什么会报错?

`List<>` 等泛型集合类要求填充的必须是**引用类型**（对象类型），而不能直接使用**基本数据类型**（如 `int`、`char`、`double` 等），否则会编译报错。

这是因为 Java 的泛型机制在设计时就只支持引用类型，不支持基本数据类型。例如，下面的代码会报错：

```
// 错误示例：List 中直接使用基本数据类型 int
List<int> list = new ArrayList<>(); // 编译报错
```

解决的办法是，使用基本数据类型对应的包装类。因此，正确的写法是：

```
// 正确示例：使用包装类 Integer
List<Integer> list = new ArrayList<>();
list.add(10); // 自动装箱：int -> Integer
int num = list.get(0); // 自动拆箱：Integer -> int
```

这么设计的原因是：

- **泛型的类型擦除机制**：Java 泛型在编译后会被擦除为 `Object` 类型，而 `Object` 只能接收引用类型，不能接收基本数据类型。
- **历史原因**：Java 最初设计时基本数据类型和引用类型是严格区分的，泛型是后期（JDK 1.5）才引入的特性，为了兼容已有的类型系统，选择只支持引用类型。

通过使用包装类，结合 Java 的**自动装箱**（基本类型 → 包装类）和**自动拆箱**（包装类 → 基本类型）机制，可以很方便地在泛型集合中操作基本数据类型的数据。

List 和数组如何互相转换?

> List 转数组

主要有两种方式，核心是用 List 的`toArray()`方法，重点注意「泛型和类型匹配」：

- 无参 toArray ()（返回 Object []，不推荐）

```
List<String> strList = new ArrayList<>();
strList.add("a");
strList.add("b");
// 返回 Object[]，强转可能报错
Object[] objArr = strList.toArray();
```

这种方式返回的是 Object 数组，若强转成 String [] 会抛 ClassCastException，仅适合不确定数组类型的场景，基本不用。

- 带参 toArray (T [] a)（推荐，指定类型）

```
List<String> strList = new ArrayList<>();
strList.add("a");
strList.add("b");
// 方式 1：传入指定长度的数组
String[] strArr1 = strList.toArray(new String[strList.size()]);
```

```
// 方式 2：传入空数组（JDK1.8+更高效）
String[] strArr2 = strList.toArray(new String[0]);


// 自定义对象 List 转数组
List<User> userList = new ArrayList<>();
userList.add(new User("张三", 20));
User[] userArr = userList.toArray(new User[0]);
```

这是最常用的方式，传入对应类型的数组，List 会把元素复制到该数组中，若传入的数组长度不足，会自动创建新数组，推荐传空数组（JDK 会优化长度）。

> 数组转 List

核心是用 `Arrays.asList()`，但要注意「返回的 List 不可变」和「基本类型数组的坑」：

- 普通对象数组转 List（常用）

```
String[] strArr = {"a", "b", "c"};
// 返回固定大小的 List（属于 Arrays 内部类，不可 add/remove）
List<String> strList1 = Arrays.asList(strArr);


// 若需要可变 List，包装一层 ArrayList
List<String> strList2 = new ArrayList<>(Arrays.asList(strArr));
strList2.add("d"); // 正常执行
```

`Arrays.asList()`返回的 List 不是 ArrayList，而是 Arrays 的内部类，不支持添加 / 删除操作，想修改就套一层 ArrayList。

- 基本类型数组转 List（避坑）

```
// 错误示例：int[]转 List 会变成 List<int[]>，而非 List<Integer>
int[] numArr = {1, 2, 3};
List<int[]> wrongList = Arrays.asList(numArr);


// 正确方式 1：手动装箱（JDK8-）
List<Integer> numList1 = new ArrayList<>();
for (int num : numArr) {
    numList1.add(num);
```

```
}

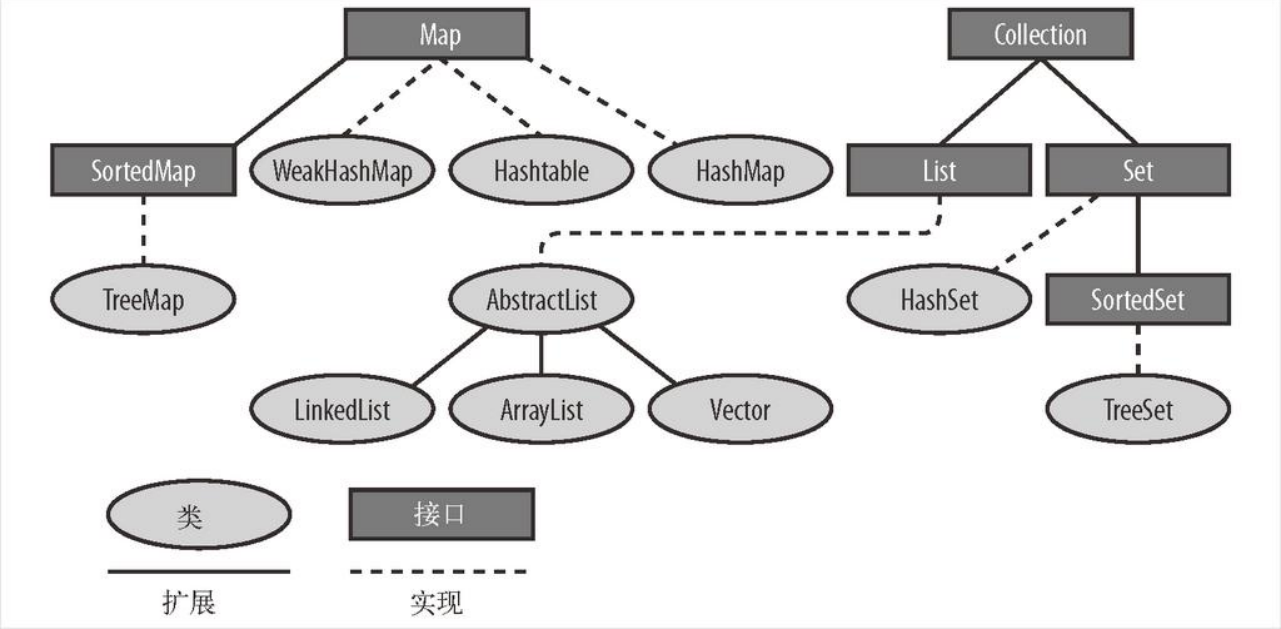

// 正确方式 2：Stream 流（JDK8+）
List<Integer> numList2 = Arrays.stream(numArr).boxed().collect(Collectors.toList());
```

基本类型数组（int []、long []）直接用`Arrays.asList()`会把整个数组当成一个元素，必须手动装箱或用 Stream 流转换为包装类（Integer）的 List。

Set

Java 集合中 List 和 Set 区别是什么?

Java 里 List 和 Set 作为 Collection 的核心子接口，最核心的区别就是「是否允许元素重复」和「是否保证有序」，原理和使用场景也因此完全不同。



先说说 List，它是有序的集合，这里的 “有序” 指的是元素的存储顺序和添加顺序一致，而且允许元素重复，甚至可以存多个 null 值。

比如往 ArrayList 里依次加 1、2、1，遍历出来还是 1、2、1，能通过下标（索引）直接访问元素，像 get (0) 就能拿到第一个元素，这是 List 独有的特性。底层实现比如 ArrayList 靠数组、LinkedList 靠双向链表，都是为了维护顺序和支持下标操作，适合需要按顺序存取、频繁根据位置访问元素的场景，比如购物车列表、订单明细这类要保留添加顺序的场景。

再看 Set，它的核心是「元素唯一」，不允许重复，最多只能存一个 null 值，而且默认不保证元素的存储顺序（除了 TreeSet、LinkedHashSet 这类特殊实现）。

比如往 HashSet 里加 1、2、1，最终只会存 1 和 2，重复的 1 会被过滤掉。Set 判断元素重复的依据是 equals () 和 hashCode () 方法（HashSet、LinkedHashSet），或者元素的自然排序 / 自定义比较器（TreeSet），它没有下标，没法通过索引访问元素，只能遍历。适合需要去重的场景，比如用户标签、商品分类、抽奖名单（避免同一个用户重复中奖）这类不允许重复元素的场景。

补充一点特殊实现的差异：List 里的 Vector 是线程安全的，但性能差，现在基本不用；Set 里的 LinkedHashSet 既保证元素唯一，又能保留添加顺序，TreeSet 则会按元素大小排序，而 ArrayList、HashSet 都是非线程安全的。

如何对 Set 排序?

Java 里 Set 本身默认不保证有序，但要实现 Set 的排序，核心是选带排序特性的 Set 实现类，或把普通 Set 转成有序结构。

TreeSet 底层是红黑树，插入时自动排序，支持「自然排序」（元素实现 Comparable）和「自定义 Comparator 排序」。

```
import java.util.TreeSet;
import java.util.Comparator;

public class SetSortDemo {
    // 1. 基本类型/字符串（自然排序）
    public static void testTreeSetBasic() {
        TreeSet<Integer> numSet = new TreeSet<>();
        numSet.add(3);
        numSet.add(1);
        numSet.add(2);
        // 遍历输出：1 2 3（自动按自然顺序升序）
        for (Integer num : numSet) {
            System.out.print(num + " ");
        }
    }

    // 2. 自定义对象（实现 Comparable 接口）
    static class User implements Comparable<User> {
        private String name;
        private int age;
```

```
    public User(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // 重写 compareTo，按年龄升序排序
    @Override
    public int compareTo(User o) {
        return this.age - o.age;
    }

    @Override
    public String toString() {
        return name + ":" + age;
    }
}

// 3. 自定义对象（传入 Comparator，按年龄降序）
public static void testTreeSetCustom() {
    TreeSet<User> userSet = new TreeSet<>(new Comparator<User>() {
        @Override
        public int compare(User u1, User u2) {
            return u2.age - u1.age; // 降序
        }
    });
    userSet.add(new User("张三", 20));
    userSet.add(new User("李四", 25));
    userSet.add(new User("王五", 22));
    // 遍历输出：李四:25 王五:22 张三:20（按年龄降序）
    for (User user : userSet) {
        System.out.println(user);
    }
}
```

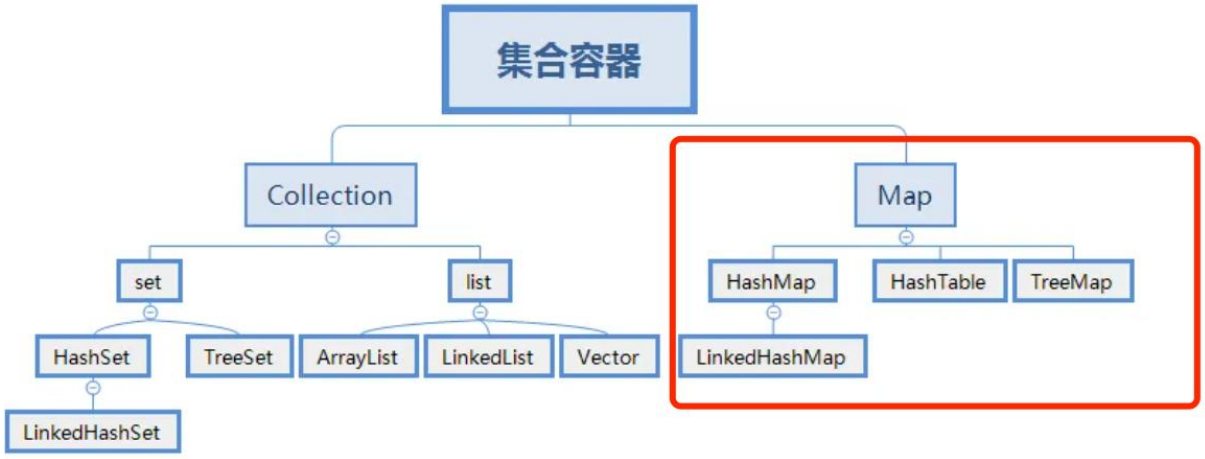
```
    // 4. LinkedHashSet：保留添加顺序（不按值排序）
    public static void testLinkedHashSet() {
        LinkedHashSet<String> strSet = new LinkedHashSet<>();
        strSet.add("b");
        strSet.add("a");
        strSet.add("c");
        // 遍历输出：b a c（和添加顺序一致）
        for (String str : strSet) {
            System.out.print(str + " ");
        }
    }
}
```

如果只是想按「插入顺序」遍历，不用按元素值排序，用 LinkedHashSet 即可，性能比 TreeSet 高：

```
import java.util.LinkedHashSet;


public class LinkedHashSetDemo {
    public static void main(String[] args) {
        LinkedHashSet<String> strSet = new LinkedHashSet<>();
        strSet.add("b");
        strSet.add("a");
        strSet.add("c");
        // 遍历输出：b a c（严格按添加顺序）
        for (String str : strSet) {
            System.out.print(str + " ");
        }
    }
}
```

Map



常见的 Map 集合（非线程安全）：

- `HashMap` 是基于哈希表实现的 `Map`，它根据键的哈希值来存储和获取键值对，JDK 1.8 中是用数组+链表+红黑树来实现的。`HashMap` 是非线程安全的，在多线程环境下，当多个线程同时对 `HashMap` 进行操作时，可能会导致数据不一致或出现死循环等问题。比如在扩容时，多个线程可能会同时修改哈希表的结构，从而破坏数据的完整性。
- `LinkedHashMap` 继承自 `HashMap`，它在 `HashMap` 的基础上，使用双向链表维护了键值对的插入顺序或访问顺序，使得迭代顺序与插入顺序或访问顺序一致。由于它继承自 `HashMap`，在多线程并发访问时，同样会出现与 `HashMap` 类似的线程安全问题。
- `TreeMap` 是基于红黑树实现的 `Map`，它可以对键进行排序，默认按照自然顺序排序，也可以通过指定的比较器进行排序。`TreeMap` 是非线程安全的，在多线程环境下，如果多个线程同时对 `TreeMap` 进行插入、删除等操作，可能会破坏红黑树的结构，导致数据不一致或程序出现异常。

常见的 Map 集合（线程安全）：

- `Hashtable` 是早期 Java 提供的线程安全的 `Map` 实现，它的实现方式与 `HashMap` 类似，但在方法上使用了 `synchronized` 关键字来保证线程安全。通过在每个可能修改 `Hashtable` 状态的方法上加上 `synchronized` 关键字，使得在同一时刻，只能有一个线程能够访问 `Hashtable` 的这些方法，从而保证了线程安全。
- `ConcurrentHashMap` 在 JDK 1.8 以前采用了分段锁等技术来提高并发性能。在 `ConcurrentHashMap` 中，将数据分成多个段（Segment），每个段都有自己的锁。在进行插入、删除等操作时，只需要获取相应段的锁，而不是整个 `Map` 的锁，这样可以允许多个线程同时

访问不同的段，提高了并发访问的效率。在 JDK 1.8 以后是通过 volatile + CAS 或者 synchronized 来保证线程安全的。

如何对 map 进行快速遍历?

- 使用 for-each 循环和 entrySet()方法：这是一种较为常见和简洁的遍历方式，它可以同时获取 `Map` 中的键和值

```
import java.util.HashMap;
import java.util.Map;

public class MapTraversalExample {
    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();
        map.put("key1", 1);
        map.put("key2", 2);
        map.put("key3", 3);

        // 使用 for-each 循环和 entrySet()遍历 Map
        for (Map.Entry<String, Integer> entry : map.entrySet()) {
            System.out.println("Key: " + entry.getKey() + ", Value: " + entry.getValue());
        }
    }
}
```

- 使用 for-each 循环和 keySet()方法：如果只需要遍历 `Map` 中的键，可以使用 `keySet()`方法，这种方式相对简单，性能也较好。

```
import java.util.HashMap;
import java.util.Map;

public class MapTraversalExample {
    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();
        map.put("key1", 1);
        map.put("key2", 2);
        map.put("key3", 3);
```

```
        // 使用 for-each 循环和 keySet()遍历 Map 的键
        for (String key : map.keySet()) {
            System.out.println("Key: " + key + ", Value: " + map.get(key));
        }
    }
}
```

- 使用迭代器：通过获取 Map 的 entrySet()或 keySet()的迭代器，也可以实现对 Map 的遍历，这种方式在需要删除元素等操作时比较有用。

```
import java.util.HashMap;
import java.util.Iterator;
import java.util.Map;
import java.util.Map.Entry;

public class MapTraversalExample {
    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();
        map.put("key1", 1);
        map.put("key2", 2);
        map.put("key3", 3);

        // 使用迭代器遍历 Map
        Iterator<Entry<String, Integer>> iterator = map.entrySet().iterator();
        while (iterator.hasNext()) {
            Entry<String, Integer> entry = iterator.next();
            System.out.println("Key: " + entry.getKey() + ", Value: " + entry.getValue());
        }
    }
}
```

- 使用 Lambda 表达式和 forEach()方法：在 Java 8 及以上版本中，可以使用 Lambda 表达式和 `forEach()`方法来遍历 `Map`，这种方式更加简洁和函数式。

```
import java.util.HashMap;
```

```
import java.util.Map;


public class MapTraversalExample {
    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();
        map.put("key1", 1);
        map.put("key2", 2);
        map.put("key3", 3);

        // 使用 Lambda 表达式和 forEach()方法遍历 Map
        map.forEach((key, value) -> System.out.println("Key: " + key + ", Value: " + value));
    }
}
```

- 使用 Stream API：Java 8 引入的 `Stream API` 也可以用于遍历 `Map`，可以将 `Map` 转换为流，然后进行各种操作。

```
import java.util.HashMap;
import java.util.Map;
import java.util.stream.Collectors;


public class MapTraversalExample {
    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();
        map.put("key1", 1);
        map.put("key2", 2);
        map.put("key3", 3);

        // 使用 Stream API 遍历 Map
        map.entrySet().stream()
          .forEach(entry -> System.out.println("Key: " + entry.getKey() + ", Value: " +
entry.getValue()));

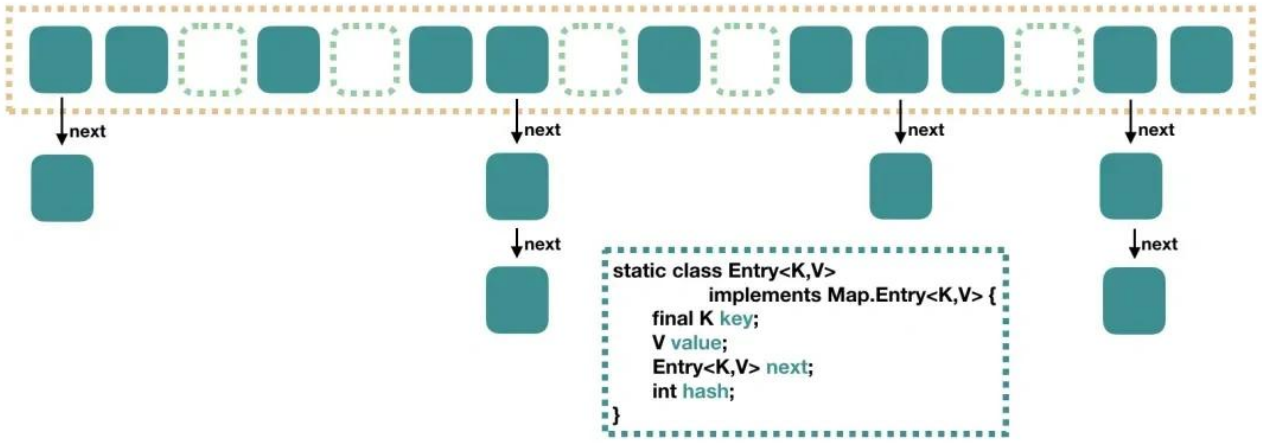
        // 还可以进行其他操作，如过滤、映射等
        Map<String, Integer> filteredMap = map.entrySet().stream()
                                            .filter(entry -> entry.getValue() > 1)
```

```
                                                .collect(Collectors.toMap(Map.Entry::getKey,
Map.Entry::getValue));
        System.out.println(filteredMap);
    }
}
```

HashMap 实现原理介绍一下?

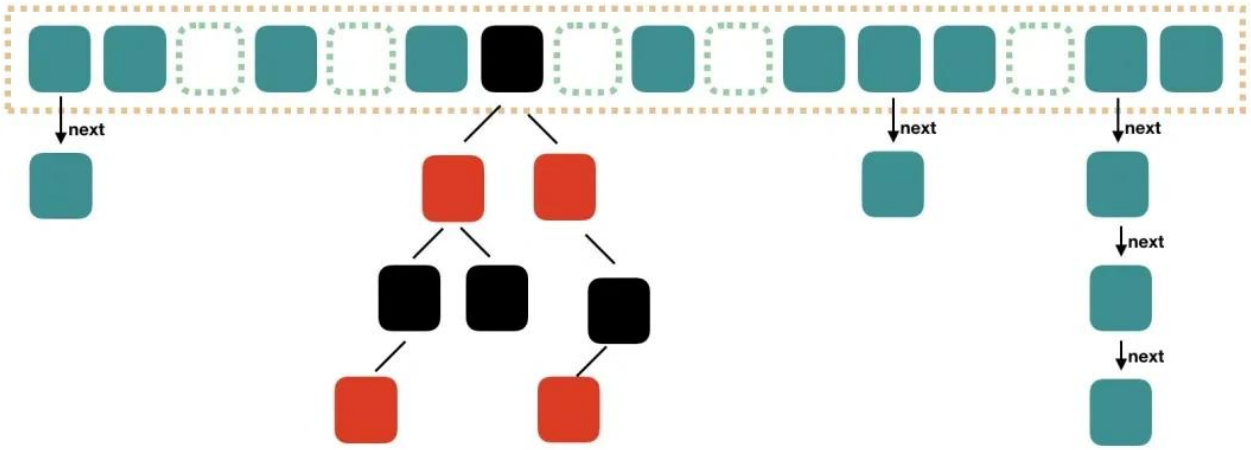
在 JDK 1.7 版本之前， HashMap 数据结构是数组和链表，HashMap 通过哈希算法将元素的键（Key）映射到数组中的槽位（Bucket）。如果多个键映射到同一个槽位，它们会以链表的形式存储在同一个槽位上，因为链表的查询时间是 O(n)，所以冲突很严重，一个索引上的链表非常长，效率就很低了。

Java7 HashMap 结构



所以在 JDK 1.8 版本的时候做了优化，当一个链表的长度超过 8 的时候就转换数据结构，不再使用链表存储，而是使用**红黑树**，查找时使用红黑树，时间复杂度 O（log n），可以提高查询性能，但是在数量较少时，即数量小于 6 时，会将红黑树转换回链表。

Java8 HashMap 结构



HashMap 链表发生转换后为什么不用平衡二叉树?

AVL 树是严格平衡的二叉树，要求任意节点的左右子树高度差不超过 1，这意味着：

- 插入 / 删除时会触发大量旋转操作（左旋、右旋、双旋），哪怕微小的高度差都要修正；
- 而 HashMap 的场景是 “链表转树” 仅发生在链表长度≥8（JDK1.8）时，本身是低频场景，为了这种低频场景付出高频的平衡开销，完全不划算；
- 红黑树仅保证黑色高度平衡（不是严格的节点高度平衡），旋转次数远少于 AVL 树，插入 / 删除的平均时间复杂度仍为 O (logn)，但实际执行效率更高。

HashMap 的核心是 「哈希 + 数组 + 链表 / 树」，树的作用只是解决哈希冲突严重导致链表过长的问题，而非做纯树形存储：

- 红黑树的**查找、插入、删除的时间复杂度都是 O (logn)**，虽然比 AVL 树的查找略慢（因为高度可能稍高），但增删的开销远低于 AVL 树；
- 对于 HashMap 来说，“增删” 和 “查找” 的频率几乎持平，红黑树的综合性能更优 ，毕竟 HashMap 不会只查不改，也不会只改不查。

了解的哈希冲突解决方法有哪些?

- 链接法：使用链表或其他数据结构来存储冲突的键值对，将它们链接在同一个哈希桶中。
- 开放寻址法：在哈希表中找到另一个可用的位置来存储冲突的键值对，而不是存储在链表中。常见的开放寻址方法包括线性探测、二次探测和双重散列。
- 再哈希法（Rehashing）：当发生冲突时，使用另一个哈希函数再次计算键的哈希值，直到找到一个空槽来存储键值对。

- 哈希桶扩容：当哈希冲突过多时，可以动态地扩大哈希桶的数量，重新分配键值对，以减少冲突的概率。

HashMap 是线程安全的吗?

hashmap 不是线程安全的，hashmap 在多线程会存在下面的问题：

- JDK 1.7 HashMap 采用数组 + 链表的数据结构，多线程背景下，在数组扩容的时候，存在 Entry 链死循环和数据丢失问题。
- JDK 1.8 HashMap 采用数组 + 链表 + 红黑二叉树的数据结构，优化了 1.7 中数组扩容的方案，解决了 Entry 链死循环和数据丢失问题。但是多线程背景下，put 方法存在数据覆盖的问题。

如果要保证线程安全，可以通过这些方法来保证：

- 多线程环境可以使用 Collections.synchronizedMap 同步加锁的方式，还可以使用 HashTable，但是同步的方式显然性能不达标，而 ConurrentHashMap 更适合高并发场景使用。
- ConcurrentHashmap 在 JDK1.7 和 1.8 的版本改动比较大，1.7 使用 Segment+HashEntry 分段锁的方式实现，1.8 则抛弃了 Segment，改为使用 CAS+synchronized+Node 实现，同样也加入了红黑树，避免链表过长导致性能的问题。

在 Java 的 hashmap 中 get 一个元素的过程是怎样的?

`get` 方法的作用是传入我们需要获取的节点的 `key`，然后将这个节点的 `value` 返回。首先先贴上 `get` 方法的代码：

```
public V get(Object key) {
    Node<K,V> e;
    return (e = getNode(hash(key), key)) == null ? null : e.value;
}
```

可以看到，`get` 方法的代码非常的简洁，因为具体的代码都封装在了 `getNode` 这个方法里面，`get` 方法只是对它进行了调用。

`getNode` 方法接收两个参数，第一个参数是 `key` 的 `hash` 值，第二个参数就是 `key` 本身。下面我们就来看看 `getNode` 方法的源代码（通过注释，对源码进行了逐句解读）：

```
/**
```

```
 * Implements Map.get and related methods
 *
 * @param hash key 到 hash 值
 * @param key key 值
 * @return the node, or null if none
 */
final HashMap.Node<K,V> getNode(int hash, Object key) {
    HashMap.Node<K,V>[] tab; HashMap.Node<K,V> first, e; int n; K k;

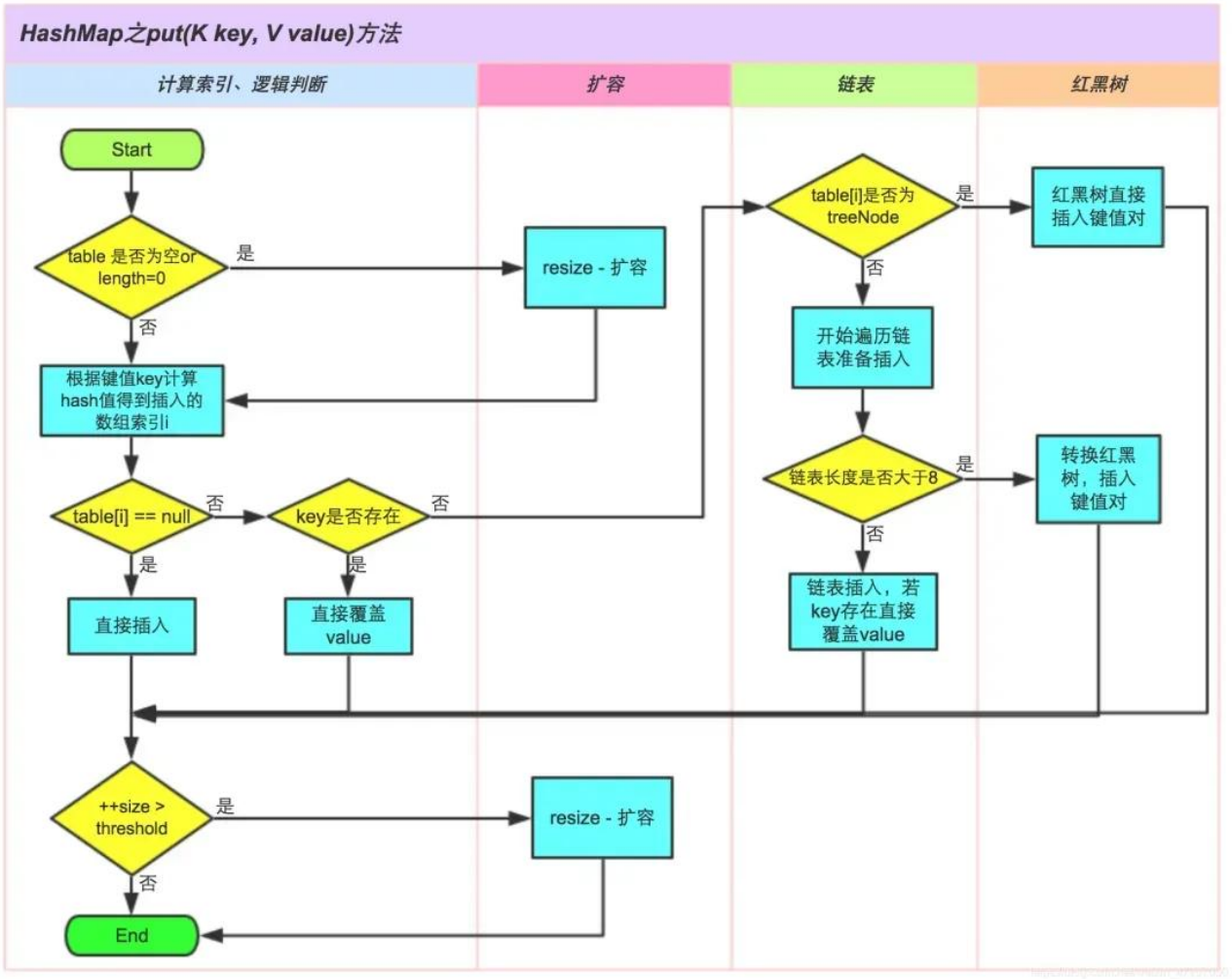
    // 以下 if 语句中判断三个条件：
    //   1、HashMap 中存储数据的数组 table 不为 null；
    //   2、数组 table 不为 null，且长度大于 0；
    //   3、table 已经创建，且通过 hash 值计算出的节点存放位置有节点存在；
    // 若上面三个条件都满足，才表示 HashMap 中可能有我们需要获取的元素
    if ((tab = table) != null && (n = tab.length) > 0 &&
        (first = tab[(n - 1) & hash]) != null) {

        // 定位到元素在数组中的位置后，我们开始沿着这个位置的链表或者树开始遍历寻找
        // 注：JDK1.8 之前，HashMap 的实现是数组+链表，到 1.8 开始变成数组+链表+红黑树

        // 首先判断这个位置的第一个节点的 key 值是否与参数的 key 值相等，
        // 若相等，则这个节点就是我们要找的节点，将其返回
        if (first.hash == hash && // always check first node
            ((k = first.key) == key || (key != null && key.equals(k))))
            return first;
        // 若上面的不满足，则判断第一个节点是否有下一个节点
        // 若有，继续判断；若没有，那表示我们要找的节点不存在
        if ((e = first.next) != null) {
            // 若第一个节点是应该树节点，则通过红黑树的查找算法进行查找
            if (first instanceof HashMap.TreeNode)
                return ((HashMap.TreeNode<K,V>)first).getTreeNode(hash, key);
            // 若不是一个树节点，表示当前位置是一个链表，则使用 do...while 循环遍历查找
            do {
                // 若查找到某个节点的 key 值与参数的 key 值相等，则表示它就是我们要找的节点，将其返回
```

```
                if (e.hash == hash &&
                    ((k = e.key) == key || (key != null && key.equals(k))))
                    return e;
            } while ((e = e.next) != null);
        }
    }
    // 若没有找到对应的节点，返回 null
    return null;
}
```

hashmap 的 put 过程介绍一下



HashMap HashMap 的 put()方法用于向 HashMap 中添加键值对，当调用 HashMap 的 put()方法时，会按照以下详细流程执行（JDK8 1.8 版本）：

> 第一步：根据要添加的键的哈希码计算在数组中的位置（索引）。

> 第二步：检查该位置是否为空（即没有键值对存在）

- 如果为空，则直接在该位置创建一个新的 Entry 对象来存储键值对。将要添加的键值对作为该 Entry 的键和值，并保存在数组的对应位置。将 HashMap 的修改次数（modCount）加 1，以便在进行迭代时发现并发修改。

> 第三步：如果该位置已经存在其他键值对，检查该位置的第一个键值对的哈希码和键是否与要添加的键值对相同？

- 如果相同，则表示找到了相同的键，直接将新的值替换旧的值，完成更新操作。

> 第四步：如果第一个键值对的哈希码和键不相同，则需要遍历链表或红黑树来查找是否有相同的键：

如果键值对集合是链表结构，从链表的头部开始逐个比较键的哈希码和 equals()方法，直到找到相同的键或达到链表末尾。

- 如果找到了相同的键，则使用新的值取代旧的值，即更新键对应的值。
- 如果没有找到相同的键，则将新的键值对添加到链表的头部。

如果键值对集合是红黑树结构，在红黑树中使用哈希码和 equals()方法进行查找。根据键的哈希码，定位到红黑树中的某个节点，然后逐个比较键，直到找到相同的键或达到红黑树末尾。

- 如果找到了相同的键，则使用新的值取代旧的值，即更新键对应的值。
- 如果没有找到相同的键，则将新的键值对添加到红黑树中。

> 第五步：检查链表长度是否达到阈值（默认为 8）：

- 如果链表长度超过阈值，且 HashMap 的数组长度大于等于 64，则会将链表转换为红黑树，以提高查询效率。

> 第六步：检查负载因子是否超过阈值（默认为 0.75）：

- 如果键值对的数量（size）与数组的长度的比值大于阈值，则需要进行扩容操作。

> 第七步：扩容操作：

- 创建一个新的两倍大小的数组。
- 将旧数组中的键值对重新计算哈希码并分配到新数组中的位置。
- 更新 HashMap 的数组引用和阈值参数。

> 第八步：完成添加操作。

此外，HashMap 是非线程安全的，如果在多线程环境下使用，需要采取额外的同步措施或使用线程安全的 ConcurrentHashMap。

HashMap 的 put(key,val)和 get(key)过程

- 存储对象时，我们将 K/V 传给 put 方法时，它调用 hashCode 计算 hash 从而得到 bucket 位置，进一步存储，HashMap 会根据当前 bucket 的占用情况自动调整容量(超过 Load Facotr 则 resize 为原来的 2 倍)。
- 获取对象时，我们将 K 传给 get，它调用 hashCode 计算 hash 从而得到 bucket 位置，并进一步调用 equals()方法确定键值对。如果发生碰撞的时候，Hashmap 通过链表将产生碰撞冲突的元素组织起来，在 Java 8 中，如果一个 bucket 中碰撞冲突的元素超过某个限制(默认是 8)，则使用红黑树来替换链表，从而提高速度。

hashmap 调用 get 方法一定安全吗?

不是，调用 get 方法有几点需要注意的地方：

- **空指针异常（NullPointerException）**：如果你尝试用 `null` 作为键调用 `get` 方法，而 `HashMap` 没有被初始化（即为 `null`），那么会抛出空指针异常。不过，如果 `HashMap` 已经初始化，使用 `null` 作为键是允许的，因为 `HashMap` 支持 `null` 键。
- **线程安全**：`HashMap` 本身不是线程安全的。如果在多线程环境中，没有适当的同步措施，同时对 `HashMap` 进行读写操作可能会导致不可预测的行为。例如，在一个线程中调用 `get` 方法读取数据，而另一个线程同时修改了结构（如增加或删除元素），可能会导致读取操作得到错误的结果或抛出 `ConcurrentModificationException`。如果需要在多线程环境中使用类似 `HashMap` 的数据结构，可以考虑使用 `ConcurrentHashMap`。

HashMap 一般用什么做 Key? 为啥 String 适合做 Key 呢?

用 string 做 key，因为 String 对象是不可变的，一旦创建就不能被修改，这确保了 Key 的稳定性。如果 Key 是可变的，可能会导致 hashCode 和 equals 方法的不一致，进而影响 HashMap 的正确性。

为什么 HashMap 要用红黑树而不是平衡二叉树?

- 平衡二叉树追求的是一种 **“完全平衡”** 状态：任何结点的左右子树的高度差不会超过 1，优势是树的结点是很平均分配的。这个要求实在是太严了，导致每次进行插入/删除节点的时候，几乎都会破坏平衡树的第二个规则，进而我们都需要通过**左旋**和**右旋**来进行调整，使之再次成为一颗符合要求的平衡树。

- 红黑树不追求这种完全平衡状态，而是追求一种 **“弱平衡”** 状态：整个树最长路径不会超过最短路径的 2 倍。优势是虽然牺牲了一部分查找的性能效率，但是能够换取一部分维持树平衡状态的成本。与平衡树不同的是，红黑树在插入、删除等操作，**不会像平衡树那样，频繁着破坏红黑树的规则，所以不需要频繁着调整**，这也是我们为什么大多数情况下使用红黑树的原因。

hashmap key 可以为 null 吗?

可以为 null。

- hashMap 中使用 hash()方法来计算 key 的哈希值，当 key 为空时，直接另 key 的哈希值为 0，不走 key.hashCode()方法；

```
static final int hash(Object key) {
    int h;
    //当key等于null的时候，不走hashCode()方法
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

- hashMap 虽然支持 key 和 value 为 null，但是 null 作为 key 只能有一个，null 作为 value 可以有多个；
- 因为 hashMap 中，如果 key 值一样，那么会覆盖相同 key 值的 value 为最新，所以 key 为 null 只能有一个。

重写 HashMap 的 equal 和 hashcode 方法需要注意什么?

HashMap 使用 Key 对象的 hashCode()和 equals 方法去决定 key-value 对的索引。当我们试着从 HashMap 中获取值的时候，这些方法也会被用到。如果这些方法没有被正确地实现，在这种情况下，两个不同 Key 也许会产生相同的 hashCode()和 equals()输出，HashMap 将会认为它们是相同的，然后覆盖它们，而非把它们存储到不同的地方。

同样的，所有不允许存储重复数据的集合类都使用 hashCode()和 equals()去查找重复，所以正确实现它们非常重要。equals()和 hashCode()的实现应该遵循以下规则：

- 如果 o1.equals(o2)，那么 o1.hashCode() == o2.hashCode()总是为 true 的。
- 如果 o1.hashCode() == o2.hashCode()，并不意味着 o1.equals(o2)会为 true。

重写 HashMap 的 equal 方法不当会出现什么问题?

HashMap 在比较元素时，会先通过 hashCode 进行比较，相同的情况下再通过 equals 进行比较。

所以 equals 相等的两个对象，hashCode 一定相等。hashCode 相等的两个对象，equals 不一定相等（比如散列冲突的情况）

重写了 equals 方法，不重写 hashCode 方法时，可能会出现 equals 方法返回为 true，而 hashCode 方法却返回 false，这样的一个后果会导致在 hashmap 等类中存储多个一模一样的对象，导致出现覆盖存储的数据的问题，这与 hashmap 只能有唯一的 key 的规范不符合。

列举 HashMap 在多线程下可能会出现的问题?

- JDK1.7 中的 HashMap 使用头插法插入元素，在多线程的环境下，扩容的时候有可能导致环形链表的出现，形成死循环。因此，JDK1.8 使用尾插法插入元素，在扩容时会保持链表元素原本的顺序，不会出现环形链表的问题。
- 多线程同时执行 put 操作，如果计算出来的索引位置是相同的，那会造成前一个 key 被后一个 key 覆盖，从而导致元素的丢失。此问题在 JDK 1.7 和 JDK 1.8 中都存在。

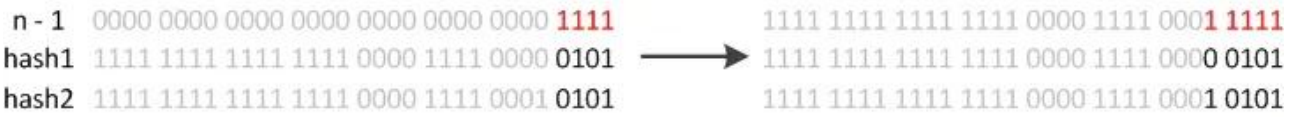
HashMap 的扩容机制介绍一下

hashMap 默认的负载因子是 0.75，即如果 hashmap 中的元素个数超过了总容量 75%，则会触发扩容，扩容分为两个步骤：

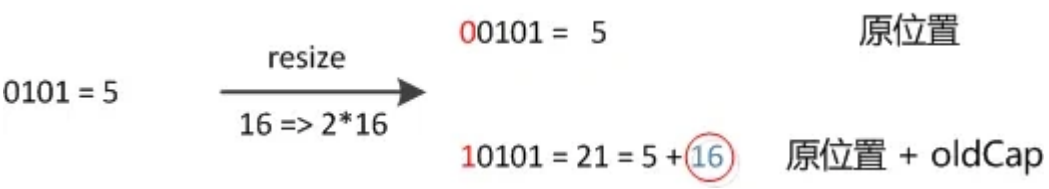
- **第 1 步**是对哈希表长度的扩展（2 倍）
- **第 2 步**是将旧哈希表中的数据放到新的哈希表中。

因为我们使用的是 2 次幂的扩展(指长度扩为原来 2 倍)，所以，元素的位置要么是在原位置，要么是在原位置再移动 2 次幂的位置。

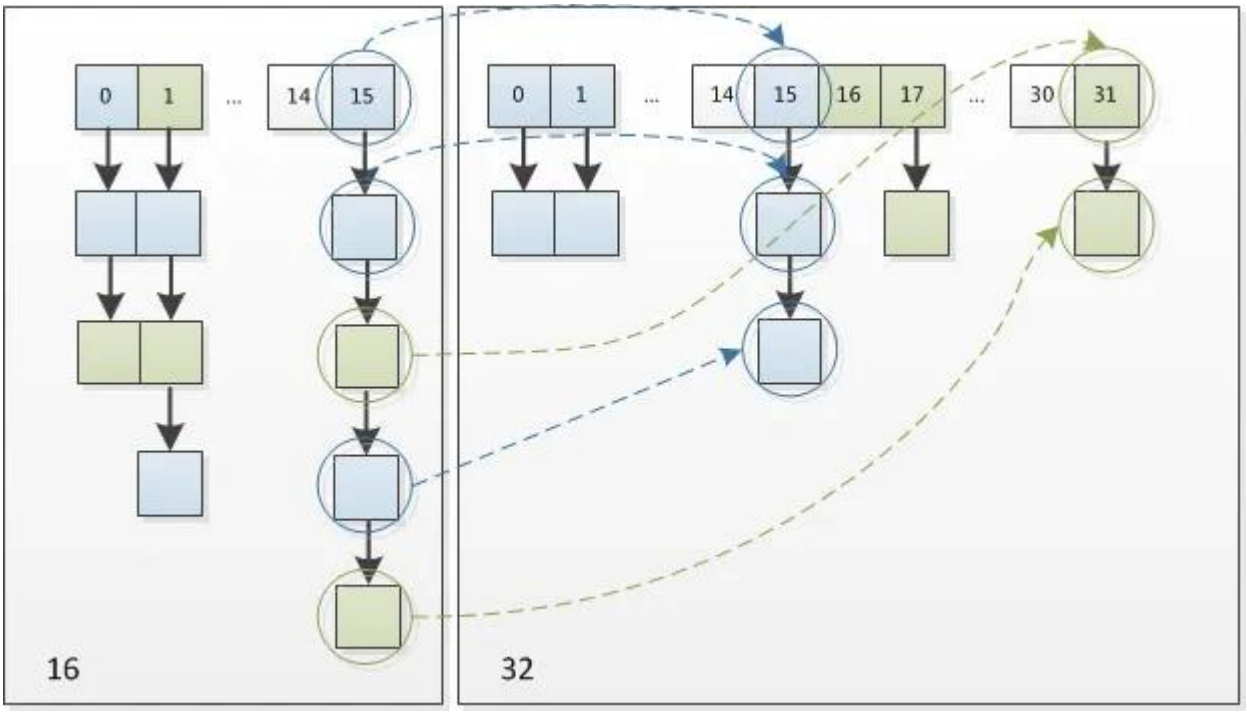
如我们从 16 扩展为 32 时，具体的变化如下所示：



因此元素在重新计算 hash 之后，因为 n 变为 2 倍，那么 n-1 的 mask 范围在高位多 1bit(红色)，因此新的 index 就会发生这样的变化：



因此，我们在扩充 HashMap 的时候，不需要重新计算 hash，只需要看看原来的 hash 值新增的那个 bit 是 1 还是 0 就好了，是 0 的话索引没变，是 1 的话索引变成“原索引+oldCap”。可以看看下图为 16 扩充为 32 的 resize 示意图：



这个设计确实非常的巧妙，既省去了重新计算 hash 值的时间，而且同时，由于新增的 1bit 是 0 还是 1 可以认为是随机的，因此 resize 的过程，均匀的把之前的冲突的节点分散到新的 bucket 了。

HashMap 的大小为什么是 2 的 n 次方大小呢?

HashMap 底层是「数组 + 链表 / 红黑树」的结构，当我们要存一个 key-value 时，第一步就是**确定这个 key 存在数组的哪个位置（索引）**。

HashMap 用的索引计算公式是：

```
索引 = hash & (length - 1)
```

这里的 `hash` 是经过扰动处理后的 key 的哈希值，`length` 是数组的容量（也就是我们说的 “大小”）。

这个公式的设计初衷是**用位运算替代取模运算**（因为位运算直接操作二进制位，速度远快于除法 / 取模），但它能生效的前提，就是 `length` 必须是 2 的 n 次方 —— 这是所有优化的基础。

下面我们逐个拆解原因，每个原因都配具体例子：

> 原因 1：保证「位运算等价于取模」，实现高效寻址

我们先看**当 length 是 2 的 n 次方时**，会发生什么：

- 假设 length = 16（即 2^4），那么 `length - 1 = 15`，二进制是 `00001111`（低 4 位全是 1）。
- 再假设 key 的 hash 值是 `100`，二进制是 `01100100`。

现在计算 `hash & (length - 1)`：

```
  01100100  (hash = 100)
& 00001111  (length-1 = 15)
= 00000100  (结果 = 4)
```

你会发现，这个结果和 `100 % 16`（取模）的结果**完全一样**，都是 4。

为什么会这样？因为当 `length` 是 2 的 n 次方时，`length - 1` 的二进制低 n 位全是 1，高位全是 0。此时做「与运算」，相当于直接把 hash 值的**低 n 位截取下来**，这在数学上就等价于「对 length 取模」。

> 反例对比：如果 length 不是 2 的 n 次方

假设 length = 15（不是 2 的 n 次方），`length - 1 = 14`，二进制是 `00001110`（最后一位是 0）。

还是用 hash = 100（二进制 `01100100`）计算：

```
  01100100  (hash = 100)
& 00001110  (length-1 = 14)
= 00000100  (结果 = 4)
```

看起来结果还行？但你再试一个 hash = 101（二进制 `01100101`）：

```
  01100101  (hash = 101)
& 00001110  (length-1 = 14)
= 00000100  (结果还是 4! )
```

发现问题了吗？因为 `length - 1` 的最后一位是 0，不管 hash 值的最后一位是 0 还是 1，与运算后都会变成 0—— 这就导致**索引的最后一位永远用不到**，比如索引 1、3、5、7... 这些位置永远不会存数据，既浪费了数组空间，又大大增加了哈希碰撞的概率（不同的 hash 挤到同一个索引里）。

> 原因 2：让哈希值的低位更均匀，减少碰撞

刚才的反例已经提到了碰撞问题，这里再深入说一下：

HashMap 会对 key 的原始 hashCode 做**扰动处理**（比如 JDK1.8 里是 `hash = (h = key.hashCode()) ^ (h >>> 16)`），目的是让 hash 值的二进制位尽可能均匀分布。

但只有当 `length - 1` 的二进制是**全 1** 时，才能 “接住” 这些均匀分布的位。比如 length=16 时，`length-1=15`（`1111`），hash 值的低 4 位每一位都能影响最终索引；如果 length=15，`length-1=14`（`1110`），最后一位直接失效，相当于少了一位来分散 hash，碰撞概率自然就高了。

> 原因 3：优化扩容时的元素重分配，不用重新算 hash

HashMap 有个扩容机制：当元素个数达到 `容量 * 负载因子`（默认是 0.75）时，数组会扩容为**原来的 2 倍**。

如果容量始终是 2 的 n 次方，扩容时元素的新索引就**不用重新计算完整的 hash**，只需要看 hash 值的**某一个高位**就行 —— 这是 JDK1.8 的核心优化之一。

我们还是用具体例子说明：

- 旧容量 oldCap = 16（2^4），二进制是 `00010000`。
- 旧索引：假设某个 key 的 hash 值是 `20`（二进制 `00010100`），旧索引是 `20 & 15 = 4`。
- 扩容后新容量 newCap = 32（2^5），现在要算新索引。

> 关键逻辑：看 hash & oldCap 的结果

旧容量 oldCap = 16（`00010000`），我们计算 `hash & oldCap`：

```
  00010100  (hash = 20)
& 00010000  (oldCap = 16)
= 00010000  (结果 = 16，不为 0)
```

- 如果结果为 **0**：说明 hash 值对应 oldCap 的那一位是 0，新索引 = 旧索引（还是 4）。
- 如果结果**不为 0**：说明那一位是 1，新索引 = 旧索引 + oldCap（4 + 16 = 20）。

你看，整个过程只需要做一次「与运算」，根本不用重新计算 hash，也不用再取模，速度非常快。而且通过这个高位判断，还能把原来挤在同一个旧索引里的元素，均匀拆分到新数组的两个索引位（旧索引和旧索引 + 旧容量），进一步降低了哈希碰撞。

> 总结

HashMap 的大小设计为 2 的 n 次方，是一个**环环相扣的优化设计**：

1. 保证 `hash & (length - 1)` 等价于取模，用位运算实现高效寻址；
2. 让 `length - 1` 的二进制全 1，接住 hash 值的均匀分布，减少碰撞；
3. 为扩容优化铺路，不用重新算 hash，仅通过高位判断就能快速确定新索引。

往 hashmap 存 20 个元素，会扩容几次?

当插入 20 个元素时，HashMap 的扩容过程如下：

初始容量：16

- 插入第 1 到第 12 个元素时，不需要扩容。
- 插入第 13 个元素时，达到负载因子限制，需要扩容。此时，HashMap 的容量从 16 扩容到 32。

扩容后的容量：32

- 插入第 14 到第 24 个元素时，不需要扩容。

因此，总共会进行一次扩容。

说说 hashmap 的负载因子

HashMap 负载因子 loadFactor 的默认值是 0.75，当 HashMap 中的元素个数超过了容量的 75% 时，就会进行扩容。

默认负载因子为 0.75，是因为它提供了空间和时间复杂度之间的良好平衡。

负载因子太低会导致大量的空桶浪费空间，负载因子太高会导致大量的碰撞，降低性能。0.75 的负载因子在这两个因素之间取得了良好的平衡。

Hashmap 和 Hashtable 有什么不一样的？Hashmap 一般怎么用?

- **HashMap 线程不安全**，效率高一点，可以存储 null 的 key 和 value，null 的 key 只能有一个，null 的 value 可以有多个。默认初始容量为 16，每次扩充变为原来 2 倍。创建时如果给定了初始容量，则扩充为 2 的幂次方大小。底层数据结构为数组+链表，插入元素后如果链表长度大于阈值（默认为 8），先判断数组长度是否小于 64，如果小于，则扩充数组，反之将链表转化为红黑树，以减少搜索时间。
- **HashTable 线程安全**，效率低一点，其内部方法基本都经过 synchronized 修饰，不可以有 null 的 key 和 value。默认初始容量为 11，每次扩容变为原来的 2n+1。创建时给定了初始容

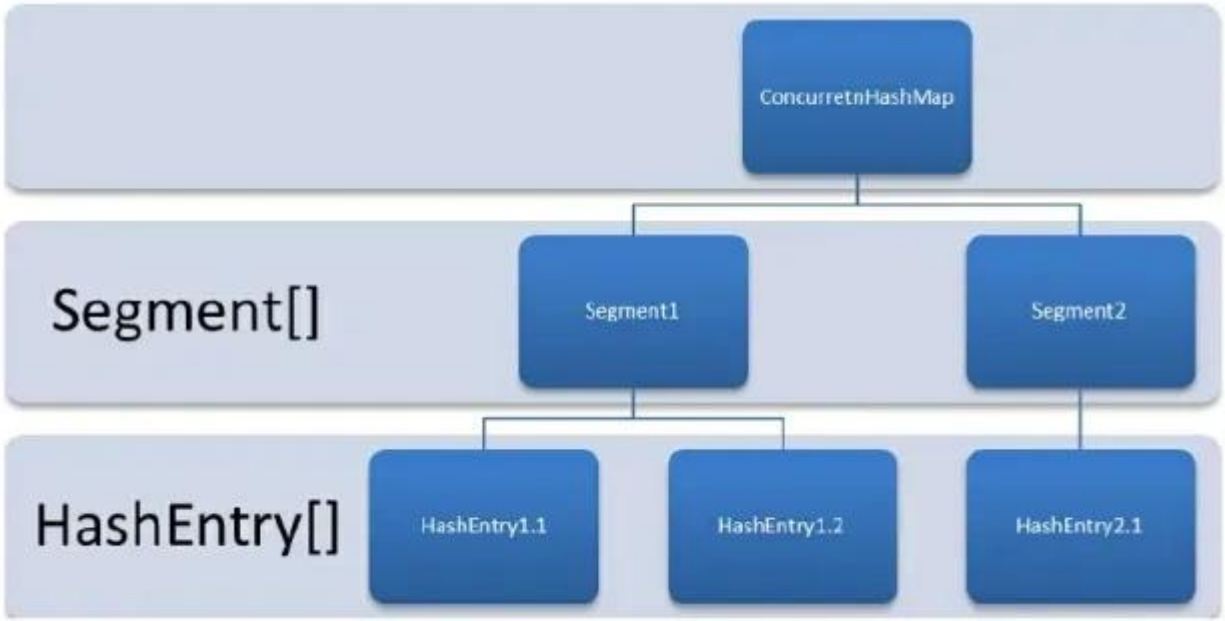
量，会直接用给定的大小。底层数据结构为数组+链表。它基本被淘汰了，要保证线程安全可以用 ConcurrentHashMap。

- **怎么用**：HashMap 主要用来存储键值对，可以调用 put 方法向其中加入元素，调用 get 方法获取某个键对应的值，也可以通过 containsKey 方法查看某个键是否存在等

ConcurrentHashMap 怎么实现的?

> JDK 1.7 ConcurrentHashMap

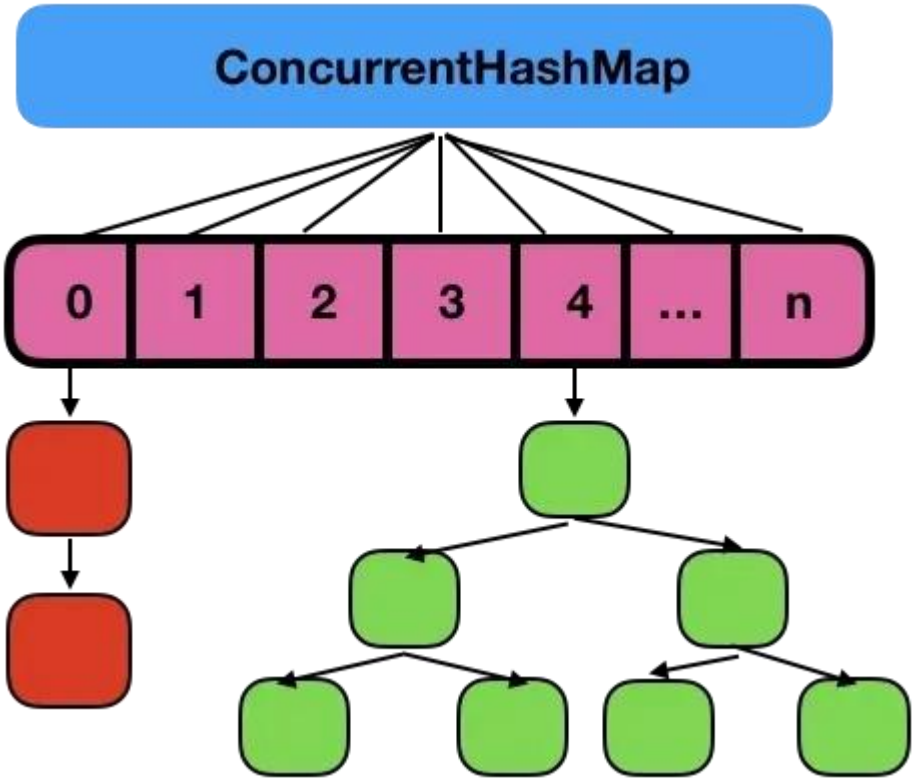
在 JDK 1.7 中它使用的是数组加链表的形式实现的，而数组又分为：大数组 Segment 和小数组 HashEntry。 Segment 是一种可重入锁（ReentrantLock），在 ConcurrentHashMap 里扮演锁的角色；HashEntry 则用于存储键值对数据。一个 ConcurrentHashMap 里包含一个 Segment 数组，一个 Segment 里包含一个 HashEntry 数组，每个 HashEntry 是一个链表结构的元素。



JDK 1.7 ConcurrentHashMap 分段锁技术将数据分成一段一段的存储，然后给每一段数据配一把锁，当一个线程占用锁访问其中一个段数据的时候，其他段的数据也能被其他线程访问，能够实现真正的并发访问。

> JDK 1.8 ConcurrentHashMap

在 JDK 1.7 中，ConcurrentHashMap 虽然是线程安全的，但因为它的底层实现是数组 + 链表的形式，所以在数据比较多的情况下访问是很慢的，因为要遍历整个链表，而 JDK 1.8 则使用了数组 + 链表/红黑树的方式优化了 ConcurrentHashMap 的实现，具体实现结构如下：



JDK 1.8 ConcurrentHashMap JDK 1.8 ConcurrentHashMap 主要通过 volatile + CAS 或者 synchronized 来实现的线程安全的。添加元素时首先会判断容器是否为空：

- 如果为空则使用 volatile 加 CAS 来初始化
- 如果容器不为空，则根据存储的元素计算该位置是否为空。
    - 如果根据存储的元素计算结果为空，则利用 CAS 设置该节点；
    - 如果根据存储的元素计算结果不为空，则使用 synchronized ，然后，遍历桶中的数据，并替换或新增节点到桶中，最后再判断是否需要转为红黑树，这样就能保证并发访问时的线程安全了。

如果把上面的执行用一句话归纳的话，就相当于是 ConcurrentHashMap 通过对头结点加锁来保证线程安全的，锁的粒度相比 Segment 来说更小了，发生冲突和加锁的频率降低了，并发操作的性能就提高了。

而且 JDK 1.8 使用的是红黑树优化了之前的固定链表，那么当数据量比较大的时候，查询性能也得到了很大的提升，从之前的 O(n) 优化到了 O(logn) 的时间复杂度。

分段锁怎么加锁的?

在 ConcurrentHashMap 中，将整个数据结构分为多个 Segment，每个 Segment 都类似于一个小的 HashMap，每个 Segment 都有自己的锁，不同 Segment 之间的操作互不影响，从而提高并发性能。

在 ConcurrentHashMap 中，对于插入、更新、删除等操作，需要先定位到具体的 Segment，然后再在该 Segment 上加锁，而不是像传统的 HashMap 一样对整个数据结构加锁。这样可以使得不同 Segment 之间的操作并行进行，提高了并发性能。

分段锁是可重入的吗?

JDK 1.7 ConcurrentHashMap 中的分段锁是用了 ReentrantLock，是一个可重入的锁。

已经用了 synchronized，为什么还要用 CAS 呢?

ConcurrentHashMap 使用这两种手段来保证线程安全主要是一种权衡的考虑，在某些操作中使用 synchronized，还是使用 CAS，主要是根据锁竞争程度来判断的。

比如：在 putVal 中，如果计算出来的 hash 槽没有存放元素，那么就可以直接使用 CAS 来进行设置值，这是因为在设置元素的时候，因为 hash 值经过了各种扰动后，造成 hash 碰撞的几率较低，那么我们可以预测使用较少的自旋来完成具体的 hash 落槽操作。

当发生了 hash 碰撞的时候说明容量不够用了或者已经有大量线程访问了，因此这时候使用 synchronized 来处理 hash 碰撞比 CAS 效率要高，因为发生了 hash 碰撞大概率来说是线程竞争比较强烈。

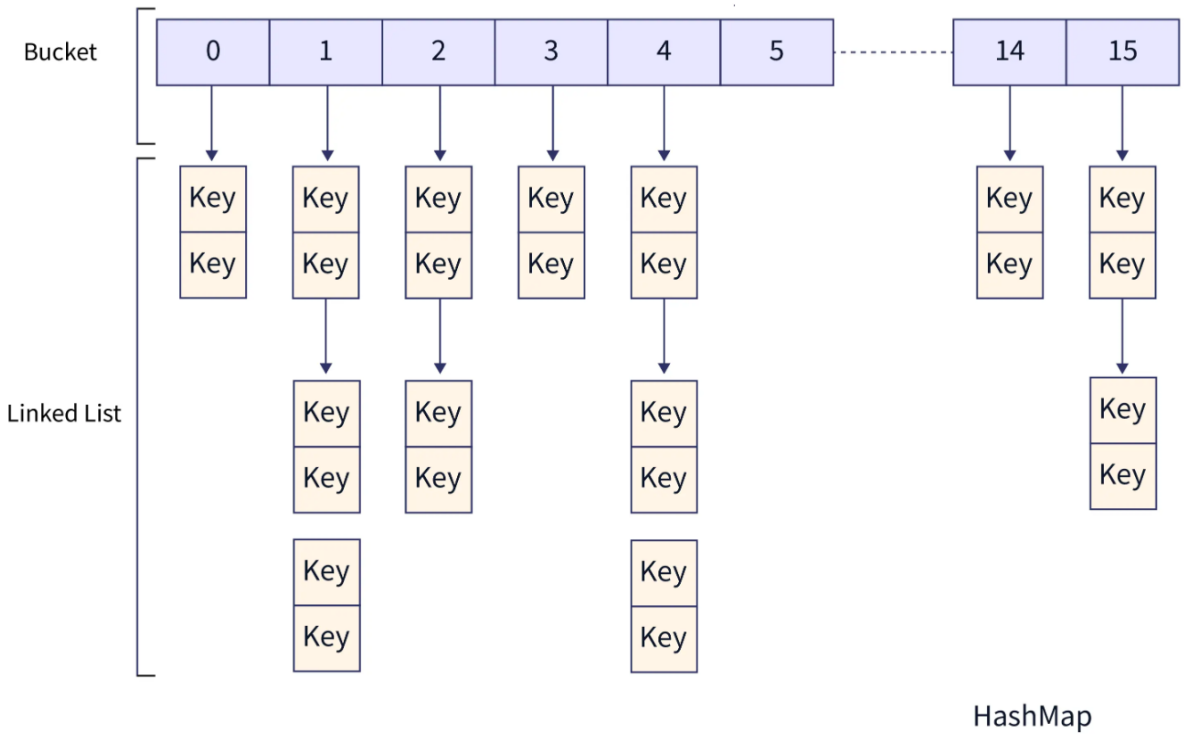
ConcurrentHashMap 用了悲观锁还是乐观锁?

悲观锁和乐观锁都有用到。

添加元素时首先会判断容器是否为空：

- 如果为空则使用 volatile 加 **CAS （乐观锁）** 来初始化。
- 如果容器不为空，则根据存储的元素计算该位置是否为空。
- 如果根据存储的元素计算结果为空，则利用 **CAS（乐观锁）** 设置该节点；
- 如果根据存储的元素计算结果不为空，则使用 **synchronized（悲观锁）** ，然后，遍历桶中的数据，并替换或新增节点到桶中，最后再判断是否需要转为红黑树，这样就能保证并发访问时的线程安全了。

HashTable 底层实现原理是什么?



- Hashtable 的底层数据结构主要是**数组加上链表**，数组是主体，链表是解决 hash 冲突存在的。
- HashTable 是线程安全的，实现方式是 **Hashtable 的所有公共方法均采用 synchronized 关键字**，当一个线程访问同步方法，另一个线程也访问的时候，就会陷入阻塞或者轮询的状态。

HashTable 线程安全是怎么实现的?

因为它的 put，get 做成了同步方法，保证了 Hashtable 的线程安全性，每个操作数据的方法都进行同步控制之后，由此带来的问题任何一个时刻**只能有一个线程可以操纵 Hashtable，所以其效率比较低**。

Hashtable 的 put(K key, V value) 和 get(Object key) 方法的源码：

```
public synchronized V put(K key, V value) {
// Make sure the value is not null
if (value == null) {
    throw new NullPointerException();
}
 // Makes sure the key is not already in the hashtable.
Entry<?,?> tab[] = table;
int hash = key.hashCode();
int index = (hash & 0x7FFFFFFF) % tab.length;
```

```
@SuppressWarnings("unchecked")
Entry<K,V> entry = (Entry<K,V>)tab[index];
for(; entry != null ; entry = entry.next) {
    if ((entry.hash == hash) && entry.key.equals(key)) {
        V old = entry.value;
        entry.value = value;
        return old;
    }
}
 addEntry(hash, key, value, index);
return null;
}

public synchronized V get(Object key) {
Entry<?,?> tab[] = table;
int hash = key.hashCode();
int index = (hash & 0x7FFFFFFF) % tab.length;
for (Entry<?,?> e = tab[index] ; e != null ; e = e.next) {
    if ((e.hash == hash) && e.key.equals(key)) {
        return (V)e.value;
    }
}
return null;
}
```

可以看到，Hashtable 是通过使用了 synchronized 关键字来保证其线程安全。

在 Java 中，可以使用 synchronized 关键字来标记一个方法或者代码块，当某个线程调用该对象的 synchronized 方法或者访问 synchronized 代码块时，这个线程便获得了该对象的锁，其他线程暂时无法访问这个方法，只有等待这个方法执行完毕或者代码块执行完毕，这个线程才会释放该对象的锁，其他线程才能执行这个方法或者代码块。

hashtable 和 concurrentHashMap 有什么区别

底层数据结构：

- jdk7 之前的 ConcurrentHashMap 底层采用的是**分段的数组+链表**实现，jdk8 之后采用的是**数组+链表/红黑树**；

- HashTable 采用的是**数组+链表**，数组是主体，链表是解决 hash 冲突存在的。

实现线程安全的方式：

- jdk8 以前，ConcurrentHashMap 采用分段锁，对整个数组进行了分段分割，每一把锁只锁容器里的一部分数据，多线程访问不同数据段里的数据，就不会存在锁竞争，提高了并发访问；jdk8 以后，直接采用数组+链表/红黑树，并发控制使用 CAS 和 synchronized 操作，更加提高了速度。
- HashTable：所有的方法都加了锁来保证线程安全，但是效率非常的低下，当一个线程访问同步方法，另一个线程也访问的时候，就会陷入阻塞或者轮询的状态。

说一下 HashMap 和 Hashtable、ConcurrentMap 的区别

- HashMap 线程不安全，效率高一点，可以存储 null 的 key 和 value，null 的 key 只能有一个，null 的 value 可以有多个。默认初始容量为 16，每次扩充变为原来 2 倍。创建时如果给定了初始容量，则扩充为 2 的幂次方大小。底层数据结构为数组+链表，插入元素后如果链表长度大于阈值（默认为 8），先判断数组长度是否小于 64，如果小于，则扩充数组，反之将链表转化为红黑树，以减少搜索时间。
- HashTable 线程安全，效率低一点，其内部方法基本都经过 synchronized 修饰，不可以有 null 的 key 和 value。默认初始容量为 11，每次扩容变为原来的 2n+1。创建时给定了初始容量，会直接用给定的大小。底层数据结构为数组+链表。它基本被淘汰了，要保证线程安全可以用 ConcurrentHashMap。
- ConcurrentHashMap 是 Java 中的一个线程安全的哈希表实现，它可以在多线程环境下并发地进行读写操作，而不需要像传统的 HashTable 那样在读写时加锁。ConcurrentHashMap 的实现原理主要基于分段锁和 CAS 操作。它将整个哈希表分成了多 Segment（段），每个 Segment 都类似于一个小的 HashMap，它拥有自己的数组和一个独立的锁。在 ConcurrentHashMap 中，读操作不需要锁，可以直接对 Segment 进行读取，而写操作则只需要锁定对应的 Segment，而不是整个哈希表，这样可以大大提高并发性能。

Set

Set 集合有什么特点？如何实现 key 无重复的?

- **set 集合特点**：Set 集合中的元素是唯一的，不会出现重复的元素。
- **set 实现原理**：Set 集合通过内部的数据结构（如哈希表、红黑树等）来实现 key 的无重复。当向 Set 集合中插入元素时，会先根据元素的 hashCode 值来确定元素的存储位置，然后再通

过 equals 方法来判断是否已经存在相同的元素，如果存在则不会再次插入，保证了元素的唯一性。

有序的 Set 是什么？记录插入顺序的集合是什么？

- **有序的 Set 是 TreeSet 和 LinkedHashSet**。TreeSet 是基于红黑树实现，保证元素的自然顺序。LinkedHashSet 是基于双重链表和哈希表的结合来实现元素的有序存储，保证元素添加的自然顺序
- **记录插入顺序的集合通常指的是 LinkedHashSet**，它不仅保证元素的唯一性，还可以保持元素的插入顺序。当需要在 Set 集合中记录元素的插入顺序时，可以选择使用 LinkedHashSet 来实现。